A Project Report on

"Development of Communication Interface Between High-Speed AXI to Low-Speed APB Peripherals Using System Verilog"

Submitted in partial fulfillment for the award of the degree of

BACHELOR OF TECHNOLOGY

in

ELECTRONICS AND COMMUNICATION ENGINEERING

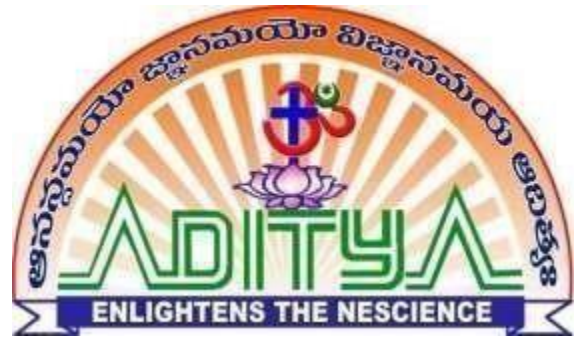
Submitted by

MUNDURI VEERA BHADRA NAGA ANUSHA
23P35A0414

Under the Esteemed Guidance of

Mr. Anjaiah Talamala M.Tech,(Ph.D)

Assistant Professor



ADITYA COLLEGE OF ENGINEERING & TECHNOLOGY
An AUTONOMOUS Institution

Approved by AICTE, Affiliated to JNTUK, Accredited by NBA, NAAC with "A+" Grade.
Recognized by UGC under the section 2(f)and 12(B)of the UGC act 1956.
Aditya Nagar, ADB Road, Surampalem-533437

(2025-2026)

ADITYA COLLEGE OF ENGINEERING & TECHNOLOGY
An AUTONOMOUS Institution
Approved by AICTE, Permanently Affiliated to JNTUK, Accredited by NBA & NAAC with A+ Grade
Recognized by UGC under Sections 2(f) and 12(B) of UGC Act, 1956
Aditya Nagar, ADB Road, Surampalem, Kakinada District - 533 437, A.P.
Ph: 99591 76665, Email: office@acet.ac.in, www.acet.ac.in

CERTIFICATE

DEPARTMENT OF ELECTRONICS AND COMMUNICATION ENGINEERING

This is to certify that the project entitled "Development of Communication Interface Between High-Speed AXI to Low-Speed APB Peripherals Using System Verilog" is being submitted by **Munduri Veera Bhadra Naga Anusha (23P35A0414)** has been carried out in the partial fulfilment of the requirements for the award of Degree of Bachelor of Technology in Electronics and Communication Engineering, Aditya College of Engineering & Technology, Surampalem affiliated to JNTUK Kakinada is a record of Bonafide work carried out for the academic year 2025-2026.

Project Guide

Mr. Anjaiah Talamala, M.Tech.,(Ph.D)

Assistant Professor

E.C.E Department

Head of the Department

Dr, R.V.V. Krishna, M.Tech.,Ph.D

Professor

E.C.E Department

EXTERNAL EXAMINER

DECLARATION

I hereby declare that the project work presented in this dissertation entitled **"Development of Communication Interface Between High-Speed AXI to Low-Speed APB Peripherals Using System Verilog"** is an original work carried out by us under the guidance of our supervisor. I further declare that this work has not been submitted, either in part or in full, for the award of any degree, diploma, or any other academic qualification in any institution or university. All the information and data used in this project have been duly acknowledged.

Yours sincerely,

Munduri Veera Bhadra Naga Anusha
23P35A0414

ACKNOWLEDGEMENT

I express our sincere gratitude to our mentor**, Mr. Anjaiah Talamala**, M.Tech.,(Ph.D). Department of Electronics and Communication Engineering, for his valuable guidance and encouragement which has been helpful in successful completion of this Project. My sincere thanks to **Dr. R.V.VKRISHNA,** M.Tech, Ph.D.**, Head of the Department** of **Electronics and Communication Engineering** for his valuable support. I feel elated to thank **Dr.A.Ramesh,** M.Tech.,Ph.D., **Principal** of Aditya College of Engineering & Technology for providing all the necessary facilities and a great support. With immense pleasure. I would like to express our deep sense and heart full thanks to the management of **Aditya College of Engineering & Technology.** I would like to thank **all the faculty members** and the **non-teaching staff members** of the Department of Electronics and Communication Engineering, directly or indirectly support for helping us in completion of this project work. Finally, I would like to thank all our **team memebers** for their continuous help and encouragement.

With Sincere Regards,

Munduri Veera Bhadra Naga Anusha

23P35A0414

ADITYA COLLEGE OF ENGINEERING & TECHNOLOGY
An AUTONOMOUS Institution
Approved by AICTE, Permanently Affiliated to JNTUK, Accredited by NBA & NAAC with A+ Grade
Recognized by UGC under Sections 2(f) and 12(B) of UGC Act, 1956
Aditya Nagar, ADB Road, Surampalem, Kakinada District – 533 437, A.P.
Ph: 99591 76665, Email: office@acet.ac.in, www.acet.ac.in

INSTITUTE VISION AND MISSION

VISION:

To induce higher planes of learning by imparting technical education with

- International standards
- Applied research
- Creative Ability
- Value based instruction and to emerge as a premiere institute

MISSION:

Achieving academic excellence by providing globally acceptable technical education by forecasting technology through

- Innovative research and development
- Industry institute interaction
- Empowered manpower

PRINCIPAL

PRINCIPAL
Aditya College of
Engineering & Technology
SURAMPALEM-533 437

ADITYA COLLEGE OF ENGINEERING & TECHNOLOGY
An AUTONOMOUS Institution
Approved by AICTE, Permanently Affiliated to JNTUK, Accredited by NBA & NAAC with A+ Grade
Recognized by UGC under Sections 2(f) and 12(B) of UGC Act, 1956
Aditya Nagar, ADB Road, Surampalem, Kakinada District - 533 437, A.P.
Ph: 99591 76663, Email: office@acet.ac.in, www.acet.ac.in

DEPARTMENT VISION AND MISSION

VISION: To emerge as a center of excellence in education and research

MISSION:

- To establish skill and learning centric infrastructure in thrust areas
- To develop Robotics and IOT based infrastructure laboratories
- To organize events through industry institute collaborations and promote innovation
- To disseminate knowledge through quality teaching learning process.

Head of the Department

Head of the Department
Dept.of ECE
Aditya College of
Engineering & Technology
SURAMPALEM-533 437

ADITYA COLLEGE OF ENGINEERING & TECHNOLOGY
An AUTONOMOUS Institution
Approved by AICTE, Permanently Affiliated to JNTUK, Accredited by NBA & NAAC with A+ Grade
Recognized by UGC under Sections 2(f) and 12(B) of UGC Act, 1956
Aditya Nagar, ADB Road, Surampalem, Kakinada District - 533 437, A.P.
Ph: 99591 76665, Email: office@acet.ac.in, www.acet.ac.in

PROGRAM SPECIFIC OUTCOMES (PSOs)

Program Name: Bachelor of Technology (B.Tech) in Electronics & Communication Engineering

PSO1: Industry ready in the arena of Electronics & Communication, VLSI, Robotics, Embedded Systems, IOT and allied fields.

PSO2: Acquire the required ability and knowledge to design, test, verify and develop innovative electronics projects through theoretical and laboratory practice.

Head of the Department

Head of the Department
Dept.of ECE
Aditya College of
Engineering & Technology
SURAMPALEM-533 437

ADITYA COLLEGE OF ENGINEERING & TECHNOLOGY
An AUTONOMOUS Institution
Approved by AICTE, Permanently Affiliated to JNTUK, Accredited by NBA & NAAC with A+ Grade
Recognized by UGC under Sections 2(f) and 12(B) of UGC Act, 1956
Aditya Nagar, ADB Road, Surampalem, Kakinada District - 533 437, A.P.
Ph: 99591 76665, Email: office@acet.ac.in, www.acet.ac.in

PROGRAM EDUCATIONAL OBJECTIVES (PEOs)

Program Name: Bachelor of Technology (B.Tech) in Electronics and Communication Engineering.

PEO1: Graduates shall evolve into skilled professionals capable of handling interdisciplinary work atmosphere and excel in problem solving.

PEO2: Graduates shall inculcate the urge to progress in the chosen field of Electronics & Communication through higher education and research.

PEO3: Graduates shall ingrain professional values through an ethics-based teaching-learning process.

PEO4: Graduates shall exhibit leadership skills and advance towards entrepreneurship, innovation and lifelong learning.

Head of the Department

Head of the Department
Dept. of ECE
Aditya College of
Engineering & Technology
SURAMPALEM-533 437

ADITYA COLLEGE OF ENGINEERING & TECHNOLOGY
An AUTONOMOUS Institution
Approved by AICTE, Permanently Affiliated to JNTUK, Accredited by NBA & NAAC with A+ Grade
Recognized by UGC under Sections 2(f) and 12(B) of UGC Act, 1956
Aditya Nagar, ADB Road, Surampalem, Kakinada District - 533 437, A.P.
Ph: 99591 76665, Email: office@acet.ac.in, www.acet.ac.in

PROGRAM OUTCOMES (POs)

PO1. **Engineering Knowledge:** Apply the knowledge of mathematics, science, engineering fundamentals, and an engineering specialization to the solution of complex engineering problems

PO2. **Problem Analysis:** Identify, formulate, research literature, and analyze complex engineering problems reaching substantiated conclusions using first principles of mathematics, natural sciences, and engineering sciences.

PO3. **Design/Development of Solutions**: Design solutions for complex engineering problems and design system components or processes that meet the specified needs with appropriate consideration for the public health and safety, and the cultural, societal, and environmental considerations.

PO4. **Conduct Investigations of Complex Problems**: Use research-based knowledge and research methods including design of experiments, analysis and interpretation of data, and synthesis of the information to provide valid conclusions.

PO5. **Modern Tool Usage**: Create, select, and apply appropriate techniques, resources, and modern engineering and IT tools including prediction and modelling to complex engineering activities with an understanding of the limitations.

PO6. **The Engineer and Society:** Apply reasoning informed by the contextual knowledge to assess societal, health, safety, legal and cultural issues and the consequent responsibilities relevant to the professional engineering practice.

PO7. **Environment and Sustainability:** Understand the impact of the professional engineering solutions in societal and environmental contexts, and demonstrate the knowledge of, and need for sustainable development.

PO8. **Ethics**: Apply ethical principles and commit to professional ethics and responsibilities and norms of the engineering practice.

PO9. **Individual and Team Work:** Function effectively as an individual, and as a member or leader in diverse teams, and in multidisciplinary settings.

PO10. **Communication:** Communicate effectively on complex engineering activities with the engineering community and with society at large, such as, being able to comprehend and write effective reports and design documentation, make effective presentations, give and receive clear instructions.

PO11. **Project Management and Finance:** Demonstrate knowledge and understanding of the engineering and management principles and apply these to one's own work, as a member and leader in a team, to manage projects and in multidisciplinary environments.

PO12. **Life-Long Learning:** Recognize the need for, and have the preparation and ability to engage in independent and life-long learning in the broadest context of technological change.

ADITYA COLLEGE OF ENGINEERING & TECHNOLOGY
An AUTONOMOUS Institution
Approved by AICTE, Permanently Affiliated to JNTUK, Accredited by NBA & NAAC with A+ Grade
Recognized by UGC under Sections 2(f) and 12(B) of UGC Act, 1956
Aditya Nagar, ADB Road, Surampalem, Kakinada District - 533 437, A.P.
Ph: 99591 76665, Email: office@acet.ac.in, www.acet.ac.in

DEPARTMENT OF ELECTRONICS AND COMMUNICATION ENGINEERING

IV B.Tech, II-SEMESTER

Course Outcomes

Up on completion of the course, students will be able to:

CO#	Course Outcomes	Blooms Taxonomy level
CO1	Identify the problem by applying acquired knowledge.	Remember
CO2	Use literature to identify the objective, scope and the Concept of the work.	Apply
CO3	Analyse and categorize executable project modules after Considering risks.	Analyse
CO4	Choose efficient tools for designing project modules.	Evaluate
CO5	Integrate all the modules through effective team work after efficient testing.	Create
CO6	Explain the completed task and compile the project report.	Understand

CO-PO/PSOMATRIX:

	PO 1	PO2	PO 3	PO 4	PO 5	PO6	PO7	PO8	PO 9	PO1 0	PO1 1	PO 12	PSO 1	PS O2
CO1	3	2	1	-	-	-	-	-	-	-	-	-	2	-
CO2	2	3	1	2	1	-	-	-	-	-	-	1	-	1
CO3	2	3	2	2	1	-	-	-	1	-	-	-	2	2
CO4	1	2	3	2	3	-	-	-	1	-	-	-	2	3
CO5	2	2	3	2	2	-	-	-	3	2	1	-	2	2
CO6	1	1	1	1	1	-	-	2	2	3	1	2	-	2
Course	1.8	2.2	1.8	2.0	1.6	-	-	2.0	1.75	2.5	2.0	1.5	2.0	2.0

Signature of the Guide

CO-PO Justification

CO NO	PO/PSO	CL	Justification
CO1	PO1	3	Applies VLSI and digital design fundamentals to understand AXI and APB protocols.
	PO2	2	Analyzes differences between high-speed AXI and low-speed APB communication.
	PO9	2	Collaborates in defining system architecture and protocol requirements.
	PO10	2	Communicates protocol concepts and system design clearly.
	PO12	2	Understands applicability in SoC and embedded systems.
	PSO1	2	Applies knowledge of RTL design and AMBA protocols.
	PSO2	2	Understands system-level integration in VLSI design.
CO2	PO1	3	Applies knowledge of digital design to develop bridge architecture.
	PO2	3	Evaluates protocol conversion mechanisms and system efficiency.
	PO4	2	Designs AXI interface, APB interface, and FSM modules.
	PO6	2	Justifies architecture using performance and design constraints.
	PO8	2	Uses SystemVerilog for modeling and simulation.
	PO9	2	Considers power and performance trade-offs in design
	PO10	2	Divides architecture design tasks within the team.
	PO12	2	Documents architecture using block diagrams and flowcharts.
	PSO1	2	Relates design to SoC applications.
	PSO2	2	Designs efficient communication systems.
CO3	PO1	2	Applies RTL design concepts for protocol conversion.
	PO2	2	Analyzes FSM behavior and protocol timing.
	PO3	3	Develops SystemVerilog RTL for AXI, APB, and FSM modules.
	PO4	3	Justifies FSM-based design for control and synchronization.

	PO5	2	Uses simulation tools (Vivado) for verification.
	PO6	2	Evaluates efficient hardware utilization and power.
	PO7	2	Designs low-power RTL modules.
	PO8	2	Ensures correct and reliable data handling.
	PO9	2	Collaborates in coding and debugging.
	PO10	2	Explains RTL design and FSM working.
	PO11	2	Documents RTL implementation and simulation results.
	PO12	2	Applies design to scalable SoC systems.
	PSO1	3	Develops optimized RTL design.
	PSO2	3	Implements real-time protocol conversion.
CO4	PO1	2	Applies verification concepts using testbenches.
	PO2	2	Analyzes waveform results for correctness.
	PO3	3	Implements verification environment in SystemVerilog.
	PO4	2	Justifies correctness through simulation results
	PO5	2	Uses Xilinx Vivado for simulation and debugging.
	PO7	2	Ensures efficient design validation.
	PO8	2	Ensures reliable and error-free operation.
	PO9	2	Works collaboratively in verification tasks
	PO10	2	Presents waveform analysis and results.
	PO11	2	Maintains simulation reports and logs.
	PO12	2	Validates design for real-world applications.
	PSO1	3	Uses verification techniques for RTL validation.
	PSO2	3	Ensures reliable system behavior.
CO5	PO1	2	Applies FPGA synthesis concepts.
	PO3	2	Validates synthesized design performance.
	PO5	2	Uses Vivado tools for synthesis and analysis.
	PO6	2	Evaluates power consumption and timing constraints.
	PO8	2	Ensures safe and reliable hardware operation.
	PO9	3	Leads synthesis and implementation tasks.
	PO10	2	Demonstrates synthesis and analysis results.

	PO11	2	Documents reports (timing, power, utilization).
	PO12	2	Applies design for FPGA-based systems.

	PSO1	3	Implements FPGA-based VLSI design.
	PSO2	3	Optimizes hardware performance and efficiency.
CO6	PO1	2	Integrates complete AXI–APB bridge system.
	PO5	2	Tests full system using simulation tools.
	PO6	2	Reviews system efficiency and scalability.
	PO8	3	Ensures correct protocol behavior and reliability.
	PO9	3	Collaborates in final system integration.
	PO10	3	Presents complete system design and results.
	PO11	3	Prepares final documentation and reports.
	PO12	2	Applies system in SoC and embedded applications.
	PSO1	3	Demonstrates complete VLSI system implementation.
	PSO2	2	Evaluates deployability in real-world systems.

Signature of the Guide

ABSTRACT

Modern System-on-Chip (SoC) designs integrate high-performance processing elements and low-speed peripheral devices that often operate in different clock domains and follow distinct communication protocols. AXI-Lite is widely used for processor-to-peripheral communication due to its simple, address-based transaction model, while the Advanced Peripheral Bus (APB) is optimized for low-power and low-bandwidth peripheral access. However, a direct communication between AXI-Lite and APB is not feasible due to the difference in their protocol structure, timing, and control mechanisms.

The project focusing on the design and verification of an AXI-Lite to APB bridge that enables an efficient and reliable communication between AXI-Lite masters and APB-based peripherals in heterogeneous SoC environment. The proposed method provides a bridge protocol adaptation by converting AXI-Lite read and write transactions into APB SETUP and ACCESS phases, while ensuring proper handshaking and response generation, for reliable communication between different clock domains, suitable synchronization methods like Clock domain crossing (CDC) is incorporated to maintain data integrity and prevent metastability.

The bridge is implemented using finite state machine (FSM) based control logic to manage transaction sequence and timing coordination between the two protocols. Functional verification is carried out using direct test cases to validate read, write operations, reset behavior, and protocol compliance. The results demonstrate that the proposed AXI-Lite to APB bridge offers a simple, scalable, and robust solution for peripheral integration in modern SoC designs, and the designs are simulated and verified using Xilinx Vivado.

INDEX

LIST OF FIGURES:

CHAPTER – 1

INTRODUCTION

1.1 Introduction to AMBA 4.0 Family Protocols

The AMBA architecture consists of five major bus protocols: Advanced System Bus (ASB), Advanced High-performance Bus (AHB), Advanced Extensible Interface (AXI), Advanced Peripheral Bus (APB), and AXI Coherency Extensions (ACE). Each of these buses is designed to serve a specific function within the system. ASB was introduced as an early system bus for basic communication, while AHB improved system performance by enabling higher bandwidth data transfers. AXI further enhanced communication by supporting high-speed data exchange with greater flexibility. In contrast, APB is designed as a simple and low-power interface used to connect peripherals such as timers, UARTs, and GPIO modules.

AXI, being one of the most widely used AMBA protocols, is further divided into three main types: AXI4, AXI4-Lite, and AXI4-Stream. AXI4 is designed for high-performance memory-mapped communication and supports burst-based data transfers, making it suitable for high-bandwidth applications. AXI4-Lite is a simplified version intended for low-bandwidth control and register access, retaining essential AXI features such as separate address and data channels while eliminating burst transfers to reduce complexity. AXI4-Stream is designed for high-speed data streaming applications and does not use address-based communication, making it ideal for continuous data transfer such as multimedia or signal processing.

On the other hand, APB is optimized for simplicity and low power consumption. It operates using a straightforward two-phase transfer mechanism, making it suitable for low-speed peripherals that do not require complex communication. Since AXI4-Lite and APB are designed with different performance objectives and communication styles, direct interaction between them is not feasible. To enable communication between high-speed AXI-based systems and low-speed APB peripherals, an AXI4-Lite to APB bridge is required. This bridge translates AXI transactions into APB-compatible operations, ensuring efficient interaction between processing units and peripheral devices while maintaining system reliability

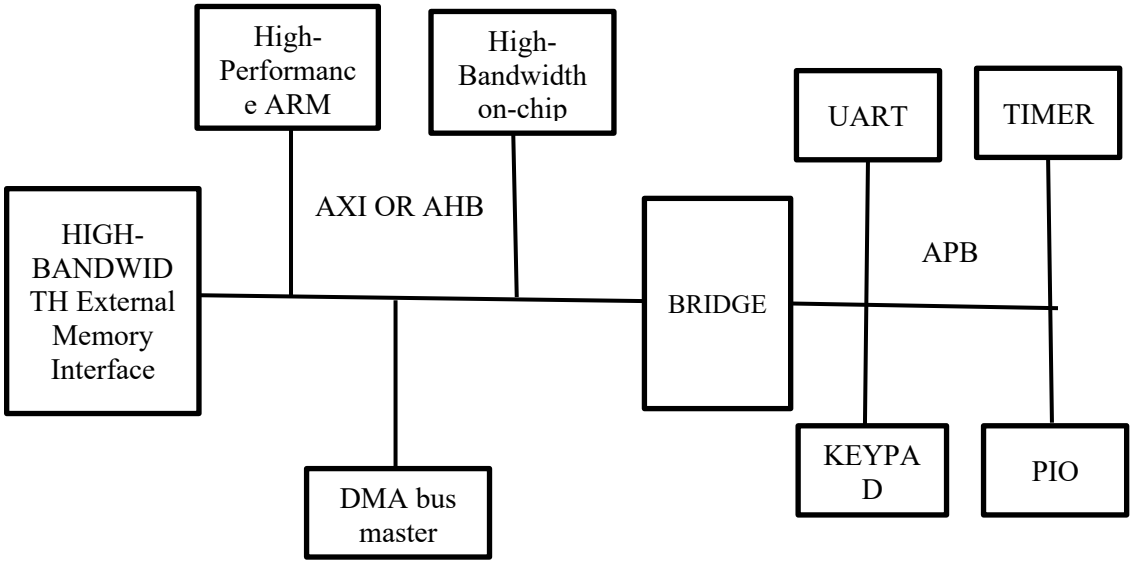


Fig 1.1 Block Diagram of SOC Architecture

The given block diagram represents a System-on-Chip (SoC) architecture designed to integrate a high-performance ARM processor with memory modules and various peripheral devices. This architecture follows a hierarchical bus structure to efficiently manage communication between components operating at different speeds. High-speed components such as the processor, on-chip memory, and external memory interface are connected through the AHB/AXI (Advanced High-performance Bus / Advanced Extensible Interface), while low-speed peripherals are connected using the APB (Advanced Peripheral Bus). A bridge is used to interface these two buses, enabling smooth communication between high-speed and low-speed subsystems.

At the core of the system is the ARM processor, which acts as the main processing unit responsible for executing instructions and controlling overall system functionality. It communicates with other components through the AHB/AXI bus, which is specifically designed for high-performance data transfers. The processor interacts with both on-chip and external memory to fetch instructions and process data. The high-bandwidth on-chip RAM provides fast and low-latency data access, making it ideal for frequently used instructions and temporary data storage. In contrast, the external memory interface allows access to larger storage devices such as DRAM or SRAM, which are essential for handling applications that require significant memory capacity.

To further enhance system performance, a Direct Memory Access (DMA) controller is included in the architecture. The DMA controller allows data transfers between memory and peripherals without continuous involvement of the processor. This significantly reduces CPU workload and improves overall system efficiency, especially during large data transfers. The AHB/AXI bus supports such high-speed operations by providing features like pipelined data transfer, burst transactions, and support for multiple bus masters, including both the processor and the DMA controller.

The AHB/AXI to APB bridge plays a critical role in connecting the high-speed and low-speed parts of the system. Since the AHB/AXI bus operates at a higher speed and uses more complex protocols compared to the APB bus, the bridge is responsible for translating transactions between these two domains. It ensures that data from the high-speed bus is properly converted into a format suitable for the APB bus, while also handling timing differences and control signals. This allows seamless communication between the processor and peripheral devices without compromising system performance.

The APB bus is designed for low-power and low-complexity communication and is mainly used to connect peripheral devices that do not require high-speed data transfer. These peripherals include components such as UART, timers, keypad interfaces, and parallel input/output (PIO) modules. The UART is used for serial communication with external devices, while the timer is used for generating delays, counting events, and managing time-based operations. The keypad provides a user input interface, and the PIO module enables communication with external hardware through parallel signals. Since these peripherals operate at lower speeds, the APB bus provides a simple and efficient interface for control and status communication.

The overall operation of the system is based on efficient coordination between high-speed and low-speed subsystems. The ARM processor executes instructions and communicates with memory and DMA through the AHB/ASB bus for high-speed data transfer. When the processor needs to interact with peripheral devices, the request is routed through the AHB/ASB to APB bridge, which converts the transaction into a format suitable for the APB bus. The peripheral then processes the request and sends the response back through the same path. Meanwhile, the DMA controller can independently handle data transfers, reducing the need for processor intervention and improving system performance.

This architecture offers several advantages, including efficient separation of high-speed and low-speed operations, reduced power consumption, improved performance through DMA support, and a scalable design that can accommodate additional peripherals. The use of a bridge ensures compatibility between different bus protocols while maintaining system efficiency. Overall, this AHB/ASB to APB-based SoC architecture provides a balanced and flexible solution for modern embedded systems, enabling high-performance processing along with efficient peripheral communication.

1.2 Problem Statement

In modern System-on-Chip (SoC) designs, integrating high-performance processing units with multiple peripheral devices presents a significant challenge due to differences in speed, complexity, and communication requirements. High-speed components such as ARM processors and memory subsystems require fast and efficient data transfer mechanisms, while peripheral devices like UART, timers, and input/output interfaces operate at much lower speeds and demand simpler communication protocols. Directly connecting these heterogeneous components without a proper interface leads to inefficiencies, increased power consumption, and potential data transfer issues.

Traditional bus architectures are not sufficient to handle both high-bandwidth and low-bandwidth communication effectively within a single system. High-speed buses like AHB or ASB are designed for performance-critical operations, but they are too complex and power-intensive for simple peripheral communication. On the other hand, low-speed buses like APB are efficient for peripheral devices but cannot support high-speed data transfers required by processors and memory. This mismatch creates a need for an optimized communication framework that can bridge the gap between these two domains. Another major issue is the increased load on the processor during data transfers between memory and peripherals. Without an efficient mechanism, the processor must handle all data movement, which reduces its availability for executing critical tasks and degrades overall system performance. Additionally, improper management of communication between different subsystems can lead to latency, reduced throughput, and poor resource utilization. The system should include a reliable bridge mechanism to interface different bus protocols and a method to reduce processor dependency during data transfers.

To address these challenges, there is a need to design an efficient bus-based architecture that ensures seamless communication between high-speed and low-speed components while maintaining performance and power efficiency. The system should include a reliable bridge mechanism to interface different bus protocols and a method to reduce processor dependency

during data transfers. Furthermore, the architecture must be scalable, allowing easy integration of additional peripherals without significantly increasing complexity. Therefore, the problem addressed in this project is the design and implementation of an optimized AHB/ASB to APB bridge-based SoC architecture that enables efficient communication between high-performance processing units and low-speed peripherals. The goal is to achieve high data transfer efficiency, reduced processor overhead, and improved overall system performance while maintaining a simple and power-efficient design suitable for embedded applications.

1.3 Motivation

In SoC-based designs, different modules operate under varying performance requirements. High-speed processors communicate using the AXI protocol, while peripheral devices such as UART, timers, and GPIO operate using simpler protocols like APB. Since these protocols differ in terms of timing, structure, and control signals, a direct interface between them is not feasible. Therefore, there is a strong need for a communication bridge that can translate AXI transactions into APB transactions. Without such a bridge, integrating low-speed peripherals into a high-speed system becomes complex and inefficient. This work addresses the need for a reliable and efficient AXI-to-APB communication interface.The AXI protocol supports high-speed data transfer with features such as pipelining, burst transactions, and multiple channels, making it suitable for processor and memory communication. In contrast, the APB protocol is designed for low-power and low-complexity operations, using a simple two-phase transfer mechanism. Due to these fundamental differences, direct communication between AXI and APB leads to mismatches in timing and data handling. The bridge plays a crucial role in resolving these differences by converting complex AXI transactions into simpler APB operations while maintaining synchronization and data integrity.

Additionally, the use of a bridge simplifies system design by providing a standardized interface between high-performance and low-power subsystems. It enables efficient integration of multiple peripherals without affecting the performance of the main processing unit. The bridge also ensures proper handling of control signals, address mapping, and data transfer, making the overall system more organized and scalable. As modern SoC designs continue to grow in complexity, the importance of such protocol conversion mechanisms becomes even more significant.

1.5 Objectives

The main objectives of the project are:

- To design an AXI-Lite to APB bridge using SystemVerilog
- To implement protocol conversion between AXI and APB
- To develop an FSM-based control mechanism for transaction handling
- To ensure correct read and write operations between master and peripherals
- To analyze synthesis reports including power, utilization, timing, and I/O ports

CHAPTER –2

LITERATURE SURVEY

2.1 Introduction

The research works related to AMBA bus protocols such as AXI, AHB, and APB, along with their design and verification techniques. These protocols are widely used in System-on-Chip (SoC) designs to enable communication between different components like processors, memory, and peripherals. Many researchers have worked on improving performance, reducing power consumption, and enhancing verification efficiency of these protocols. Different methodologies such as Verilog, SystemVerilog, UVM (Universal Verification Methodology), and VMM are used for designing and verifying these systems. The survey also highlights the importance of reusable verification components, optimized architectures, and efficient bridge designs (like AXI to APB or AHB to APB) in modern SoC development.

The existing methods have been proposed for the design and optimization of AMBA-based systems. Some researchers focused on improving AXI protocol performance by reducing delay and increasing throughput using techniques like counter-based mechanisms and optimized signal handling. Others worked on designing efficient APB bridges with additional features such as reset controllers, UART modules, and security elements like LFSR to reduce power consumption and area. Comparative studies between AMBA versions (like AMBA 2 and AMBA 3) also highlight improvements in efficiency, showing better performance in newer designs.

[1] N. Deshpande and R. Sadakale, et al. in their title "AMBA AHB to APB Bridge Protocol Verification Using SystemVerilog", the authors verify the conversion of AHB transactions into APB setup and access phases using assertions and functional checks. The work demonstrates reliable protocol translation and ensures proper transaction handling between high-performance and peripheral buses.

[2] S. Saravanan, K. G. Devika, et al. in their title "Verification of AMBA APB Bus Protocol Using UVM", the authors present a UVM-based verification methodology for the APB protocol. The work develops reusable verification components such as master and slave agents and validates important APB signals including PSEL, PENABLE, PREADY, PRDATA, and PWDATA. This structured verification approach improves test coverage and supports accurate validation of peripheral communication.

[3] N. Gaikwad and V. N. Patil, et al. in their title "Verification of AMBA AXI On-chip Communication Protocol", the authors verify the AXI protocol using a SystemVerilog-based testbench. The study validates read and write operations through VALID–READY handshake mechanisms and uses assertions and functional coverage to improve verification accuracy. The approach ensures correct channel-level communication and enhances protocol reliability.

[4] C. Ma, Z. Liu, and X. Ma, et al.in their title "Design and Implementation of APB Bridge Based on AMBA AXI 4.0", the authors propose an AXI-to-APB bridge using Verilog HDL. The bridge enables seamless communication between high-speed AXI systems and low-speed APB peripherals by converting transactions between the two protocols. The work demonstrates protocol compatibility and supports effective interaction between different performance domains within an SoC.

[5] J. Mukunthan, A. Daniel Raj, et al.in their title "Design and Implementation of AMBA APB Protocol", the authors focus on designing and implementing the APB protocol for SoC systems. The work establishes communication between master (testbench) and slave (design) using Verilog HDL. The study includes multiple test cases such as single and multiple read/write operations with and without wait states. The design is verified using a Verilog testbench and simulated in the QuestaSim tool.

[6] G. Pan, L. Luo, G. Ou, et al. is their title "Design and Verification of a MAC Controller Based on AXI Bus", presents the design of a MAC controller using the AXI bus architecture. The system integrates original design modules and reusable IP cores. The authors use a reusable verification platform based on the VMM methodology. Directed random testing and assertion-based verification is used to verify the design effectively. The SoC chip has been successfully verified and tape-out is completed.

[7] V. H. Vinay and B. S. Balaji, et al. in their title "Design of Reusable VIP (Verification IP) of AXI Master and Slave in UVM", presents the development of reusable Verification IP (VIP) for AXI Master and Slave interfaces using the UVM methodology. The VIP is designed to be configurable so that it can support different AXI-based designs. The work supports multiple AXI configurations such as different burst modes, data widths, and transaction types. The reusable components like master and slave agents, sequence libraries, and coverage models help reduce verification time and improve coverage in AXI-Lite and AXI-MM based systems.

[8] A. Gollapudi, A. Krishna Kumar, et al. in their title "Implementation of an Optimized AXI using Verilog", presents an optimized AXI protocol design to improve throughput and reduce delay in SoC systems. The proposed model allows communication between master and slave using AXI signals such as VALID, READY, and RESPONSE. To reduce the delay caused when the master waits for the slave's response, the authors introduce a counter-based mechanism. The counter decreases after each successful transaction, allowing the master to start the next transaction without waiting for the slave response. This approach improves the overall performance of the system.

[9] M. Prasanna Deepu, R. Dhanaba, et al. in their title "Validation of Transactions in AXI Protocol using System Verilog", focuses on verifying AXI protocol transactions in complex SoC designs. The paper presents a validation approach using SystemVerilog to check the communication between integrated IP components. The verification covers all five AXI channels: read address, read data, write address, write data, and write response. This method helps ensure that AXI transactions follow the protocol correctly and improves debugging and verification efficiency.

[10] K. Rawat, K. Sahni, et al. in their title "Design of AMBA APB Bridge with Reset Controller for Efficient Power Consumption", presents the design of an APB bridge with a reset controller to reduce power consumption in SoC systems. The bridge is implemented using the Verilog HDL language. The reset controller introduces a power-on reset signal to prevent metastability and avoid glitches during system startup. Simulation and synthesis are performed using ModelSim and Xilinx tools. The results show that the proposed design reduces overall power consumption compared to a bridge without power-on reset.

[11] A. Paunikar, R. Gavankar, et al. in their title "Design and Implementation of Area Efficient, Low Power AMBA-APB Bridge for SoC", presents the design of an APB bridge for SoC applications. In this work, a UART module is used as an APB slave, and a Linear Feedback Shift Register (LFSR) is included to improve data security. The paper also compares APB bridge designs based on AMBA 2 and AMBA 3 specifications in terms of power and area. The results show that the AMBA 3 APB design reduces power by about 6% and area by about 10% compared to the AMBA 2 APB design.

[12] O. Prakash, M. P. R. Prasad, **et al.** in their title "Design and Verification of Advanced Peripheral Bus (APB) RAM using SystemVerilog" presents the design and verification of an APB-based RAM module for SoC systems. The RAM supports read and write communication

between APB master and slave. Verification is performed using SystemVerilog with random testing and functional coverage. The design is simulated using Xilinx Vivado and Aldec Riviera-Pro

[13] I. H. Shanavas, M. M. Baburaj, et al. in their title "Design and Analysis of APB and AHB Lite Protocol", explains the communication between master and slave modules using AMBA protocols. The study focuses on APB and AHB-Lite, where APB uses a non pipelined approach and AHB-Lite supports higher performance. The design and verification are implemented using Verilog and testbench simulation.

[14] I. H. Shanavas, P. Akhil Reddy, **et al.** in their title "A Study on Verification of APB Protocol" (2024) surveys different methods used to design and verify the APB protocol in SoC systems. It explains how APB connects low-speed peripherals with high-speed processors using the AMBA bus architecture. The paper reviews various verification approaches using tools like SystemVerilog and UVM. It also reports high verification results, achieving about 100% functional coverage and 97% code coverage.

[15] G. Kumar, D. S. Yadav, et al. in their title "Design and Implementation of APB Interfaces for Complex SoC Environment" focuses on developing an AMBA-APB bridge for efficient communication in SoC systems. The bridge connects high-performance and low power components and improves data transfer between master and slave modules. The design optimizes resource usage and communication efficiency. Results show that the APB bridge enhances overall SoC performance and scalability.

2.2 Existing Methodology:

The AHB to APB bridge is a fundamental component in AMBA (Advanced Microcontroller Bus Architecture) based System-on-Chip (SoC) designs, enabling communication between high-performance system buses and low-power peripheral buses. The Advanced High-performance Bus (AHB) is designed for high-speed, high-bandwidth operations and supports features such as pipelining, burst transfers, and multiple masters. In contrast, the Advanced Peripheral Bus (APB) is optimized for low-power, low-complexity peripheral communication and operates without pipelining, using a simple two-phase transfer mechanism. Due to these architectural differences, direct communication between AHB and APB is not feasible, necessitating the use of a bridge.

The AHB to APB bridge functions as an AHB slave interface and an APB master interface. On the AHB side, it receives signals such as HADDR (address), HWRITE (read/write control), HWDATA (write data), HTRANS (transfer type), and HREADY (transfer completion signal). These signals are captured and interpreted by the bridge logic. On the APB side, the bridge generates corresponding control and data signals, including PADDR, PWRITE, PWDATA, PSEL (peripheral select), PENABLE (enable signal), and PRDATA (read data). The bridge ensures proper timing and synchronization between the two protocols.

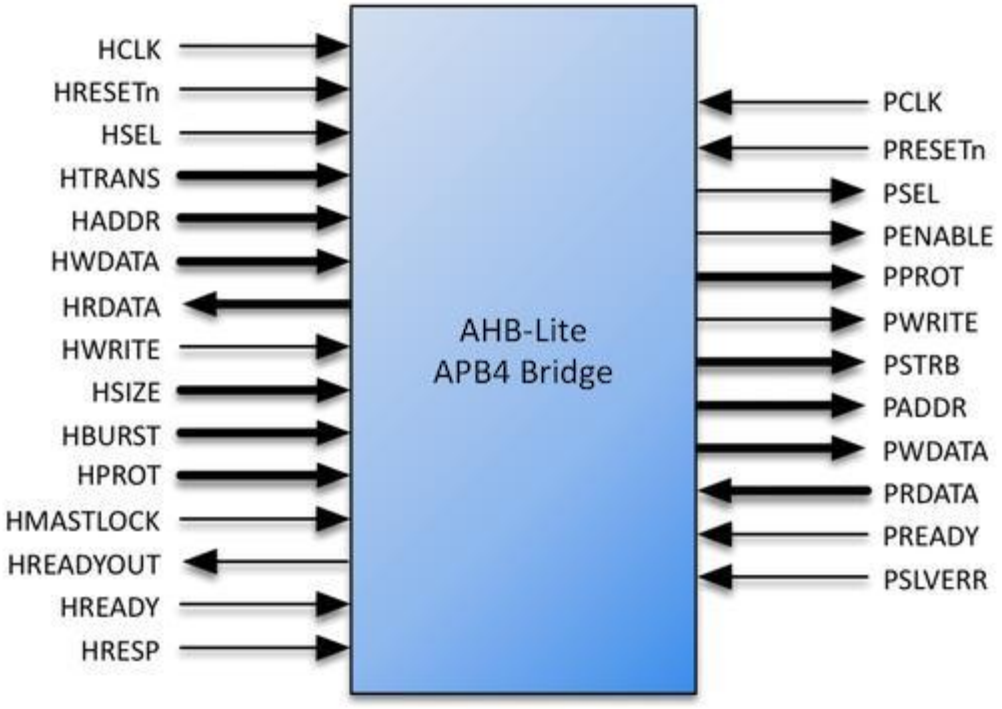


Fig-2.1 Block diagram of AHB-APB Bridge

The internal architecture of the bridge consists of three major components: address decoder, control unit (FSM), and data path logic. The address decoder is responsible for selecting the appropriate APB slave device based on the incoming AHB address. This allows multiple peripherals such as UART, timers, or GPIO modules to be connected to the APB bus. The control unit, implemented as a Finite State Machine (FSM), manages the sequence of operations required for protocol conversion. It typically includes states such as IDLE, SETUP, and ACCESS. In the SETUP phase, the bridge asserts the PSEL signal and prepares the address

and control signals. In the ACCESS phase, the PENABLE signal is asserted, and the actual data transfer takes place. The data path logic handles the movement of data between the AHB and APB interfaces. During a write operation, the bridge captures write data from the AHB bus and forwards it to the selected APB slave.

During a read operation, the bridge receives data from the APB slave and sends it back to the AHB master. Since AHB supports burst transfers while APB supports only single transfers, the bridge splits burst transactions into multiple sequential APB accesses, ensuring compatibility between the two protocols.Another important aspect of the AHB to APB bridge is timing and synchronization. AHB operates with a high-frequency clock (HCLK), whereas APB may operate on a lower-frequency clock (PCLK). The bridge must ensure proper synchronization between these clock domains to avoid timing violations. It also introduces wait states in AHB transactions when the APB slave is not ready, ensuring correct data transfer without loss.

2.3 Disadvantages of Existing Methodology

- **Increased Latency**

The AHB bus supports high-speed and pipelined data transfers, whereas the APB bus follows a simple sequential transfer mechanism. Due to this difference, each AHB transaction must be converted into a slower APB transaction, which introduces additional wait states. This results in increased latency, especially when multiple transfers are involved.

- **No Pipelining Support in APB**

AHB allows pipelined operations where multiple transactions can overlap, improving performance. However, APB does not support pipelining and processes only one transfer at a time. Because of this limitation, the bridge cannot fully utilize the high-speed capability of AHB, leading to reduced efficiency.

- **Burst Transfer Limitation**

AHB supports burst transfers where multiple data transfers occur in a single transaction. In contrast, APB supports only single data transfers. Therefore, burst transactions from AHB must be divided into multiple individual APB transfers, which increases the number of cycles required and reduces overall performance.

- **Power Throughput**

Since APB is designed for low-speed peripheral communication, the overall data transfer rate is significantly lower compared to AHB. When the bridge connects these two buses, the system performance is limited by the slower APB side, resulting in reduced throughput.

- **Additional Control Complexity**

The bridge requires additional components such as a Finite State Machine (FSM), address decoder, and control logic to manage protocol conversion. These components increase the design complexity and require careful implementation and verification to ensure correct operation.

- **Clock Domain Synchronization Issues**

In many systems, AHB and APB operate on different clock frequencies (HCLK and PCLK). The bridge must handle proper synchronization between these clock domains to avoid timing errors, metastability, or data corruption, which adds to the design challenges.

- **Power vs Performance Trade-off**

While APB is optimized for low power consumption, it sacrifices performance. Integrating it with a high-speed AHB system through the bridge creates a trade-off where power efficiency is improved, but system speed and responsiveness are reduced.

CHAPTER – 3

PROPOSED SYSTEM

3.1 Introduction

The methodology of the AXI to APB bridge design explains the step-by-step approach used to convert high-speed AXI transactions into low-speed APB transactions. In System-on-Chip (SoC) designs, different modules operate at different speeds and complexities. The AXI protocol supports high-performance, pipelined, and burst-based communication, while the APB protocol is simple, non-pipelined, and mainly used for low-power peripheral devices. Because of these differences, a proper methodology is required to ensure smooth and correct data transfer between these two protocols.

In this project, a modular design approach is followed, where the system is divided into smaller blocks such as AXI interface, FSM controller, and APB interface. Each block performs a specific function and together they achieve protocol conversion. The AXI interface handles incoming transactions, the FSM controller manages the control flow, and the APB interface generates the required APB signals. This structured approach improves design clarity, reusability, and ease of debugging.

The methodology also focuses on maintaining data integrity, reducing latency, and ensuring correct synchronization between different stages of communication. A Finite State Machine (FSM) is used as the core control unit to manage different phases like idle, setup, and enable. This makes the design more organized and ensures that each transaction follows the correct sequence according to protocol rules.

3.2 Description Of Proposed System

The proposed system is an AXI to APB bridge designed to enable efficient communication between high-speed AXI-based systems and low-speed APB peripherals. In modern embedded and SoC designs, different modules operate at varying speeds and require different communication protocols. The AXI protocol is widely used for high-performance, high-bandwidth applications due to its support for pipelined and parallel transactions, whereas the APB protocol is optimized for simple, low-power peripheral communication. Direct interaction between these two protocols is not feasible due to their differences in complexity, timing, and structure. Therefore, the proposed bridge provides a reliable and efficient

mechanism to translate AXI transactions into APB-compatible operations. The system architecture of the AXI to APB bridge is structured in a modular manner to ensure clarity, scalability, and ease of implementation. As illustrated in the block diagram, the architecture consists of five major components: AXI Master, AXI Lite Slave Interface, FSM Controller, APB Master Interface, and APB Slave. Each block performs a specific function and collectively ensures seamless data transfer between the AXI and APB domains.

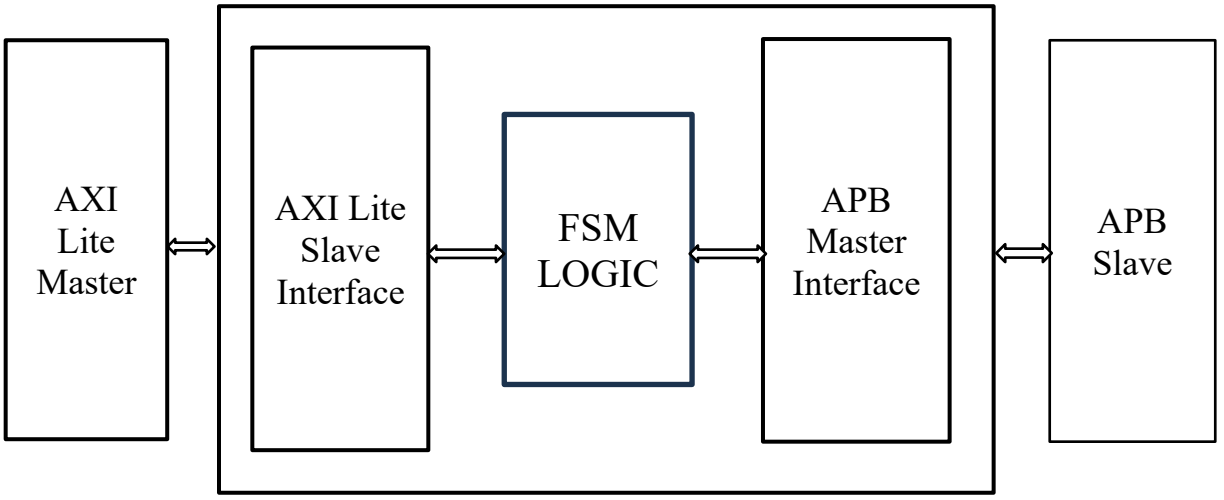


Fig3.1-Block diagram of AXI to APB Bridge

The data transfer process begins at the AXI Master, which initiates communication by sending address, control signals, and data. The AXI protocol uses separate channels for read and write operations, including address, data, and response channels. This multi-channel architecture enables high-speed and parallel data transfer, but it also introduces complexity in handling and synchronization. The AXI Lite Slave Interface serves as the entry point of the bridge and is responsible for receiving these signals from the AXI Master. Unlike full AXI, AXI Lite supports simpler transactions without burst transfers, making it suitable for control-oriented communication.

The AXI Lite Slave Interface decodes incoming signals such as read/write addresses, valid signals, and data inputs. It ensures proper handshaking using AXI signals like VALID and READY, thereby maintaining synchronization between the master and the bridge. This module effectively separates and organizes the incoming transactions, converting them into a simplified format that can be processed by the internal control logic. It also temporarily stores address and data information until the system is ready to proceed with the next stage of operation. Once the AXI signals are processed, they are forwarded to the FSM Controller, which acts as the central control unit of the entire bridge. The FSM (Finite State Machine) determines the sequence of operations required to perform a successful transaction. It typically

operates through a set of states such as idle, setup, and enable. In the idle state, the system waits for a valid transaction request. When a request is detected, the FSM moves to the setup state, where it prepares the necessary control signals for the APB interface. It then transitions to the enable state, where the actual data transfer occurs.

The FSM Controller plays a crucial role in ensuring proper coordination between the AXI and APB interfaces. It generates control signals required for APB communication and ensures that timing constraints are met. It also handles both read and write operations by generating appropriate control signals based on the type of transaction. Additionally, the FSM ensures that no data is lost during the transfer by maintaining proper synchronization and sequencing of signals. This makes the controller the "heart" of the bridge, as it governs the entire data flow process. After processing in the FSM Controller, the signals are passed to the APB Master Interface. This module is responsible for converting the processed AXI signals into APB-compatible signals. The APB protocol uses a simpler structure with signals such as PADDR (address), PWRITE (read/write control), PSEL (slave select), PENABLE (enable signal), and PWDATA (write data). The APB Master Interface generates these signals based on the inputs received from the FSM.

Since APB does not support pipelining or multiple outstanding transactions, the interface ensures that all operations are performed sequentially. It simplifies the complex AXI transactions into a format that is easily understood by APB peripherals. The interface also ensures that control signals are asserted and de-asserted at the correct time, maintaining proper protocol compliance. This conversion process is essential to ensure compatibility between the two bus systems. Finally, the APB Slave represents the peripheral device that performs the actual operation, such as reading or writing data. Examples of APB slaves include UART, timers, GPIO, and other low-speed devices. Once the APB Master Interface sends the required signals, the APB Slave processes the request and generates a response. This response is then sent back through the APB interface, passed to the FSM Controller, and finally returned to the AXI Master through the AXI Interface. This completes one full transaction cycle.

The overall design ensures efficient and reliable data transfer between two different bus protocols while maintaining simplicity and performance. By separating the system into well-defined modules, the architecture achieves modularity and scalability, allowing additional features or peripherals to be easily integrated in the future. The use of an FSM-based control mechanism ensures proper sequencing and synchronization, while the interface modules handle protocol conversion effectively. In conclusion, the proposed AXI to APB bridge provides a practical and efficient solution for connecting high-speed AXI systems with low-speed APB peripherals. It reduces system complexity, improves data handling efficiency.

3.3 Description Of AXI Lite Protocol

The Advanced Extensible Interface (AXI) is a high-performance communication protocol developed as part of the AMBA architecture to support efficient data transfer between different components in a system. It is designed to improve system performance by allowing independent read and write operations through separate channels. This separation enables parallel processing of transactions, which increases throughput and reduces communication delays between master and slave devices.

AXI follows a structured communication approach using address, data, and response channels. These channels help in organizing read and write transactions in a systematic manner. The protocol is commonly used for memory-mapped communication in processor-based systems due to its flexibility and ability to handle high-speed data exchange. A key feature of AXI is its handshake mechanism, which ensures reliable data transfer between communicating components. This mechanism is based on the use of VALID and READY signals. The VALID signal is generated by the sender to indicate that valid data or address information is available, while the READY signal is asserted by the receiver to show that it is prepared to accept the transfer. A transaction occurs only when both VALID and READY signals are active simultaneously, ensuring proper synchronization and preventing data loss.

3.3.1 Write Operation in AXI

The write operation in AXI is carried out using three independent channels: the Write Address Channel, Write Data Channel, and Write Response Channel. First, the master sends the write address and control information through the write address channel using signals like AWADDR and AWVALID. The slave acknowledges this using AWREADY. At the same time or after this, the master sends the actual data through the write data channel using WDATA and WVALID signals.

The slave receives the write data when WREADY is asserted, and for burst transfers, multiple data transfers can occur continuously. After successfully receiving the data, the slave sends a response back to the master through the write response channel using signals like BRESP and BVALID. The master acknowledges this response using BREADY. This three-channel mechanism allows address, data, and response phases to operate independently, improving performance and enabling efficient high-speed communication.

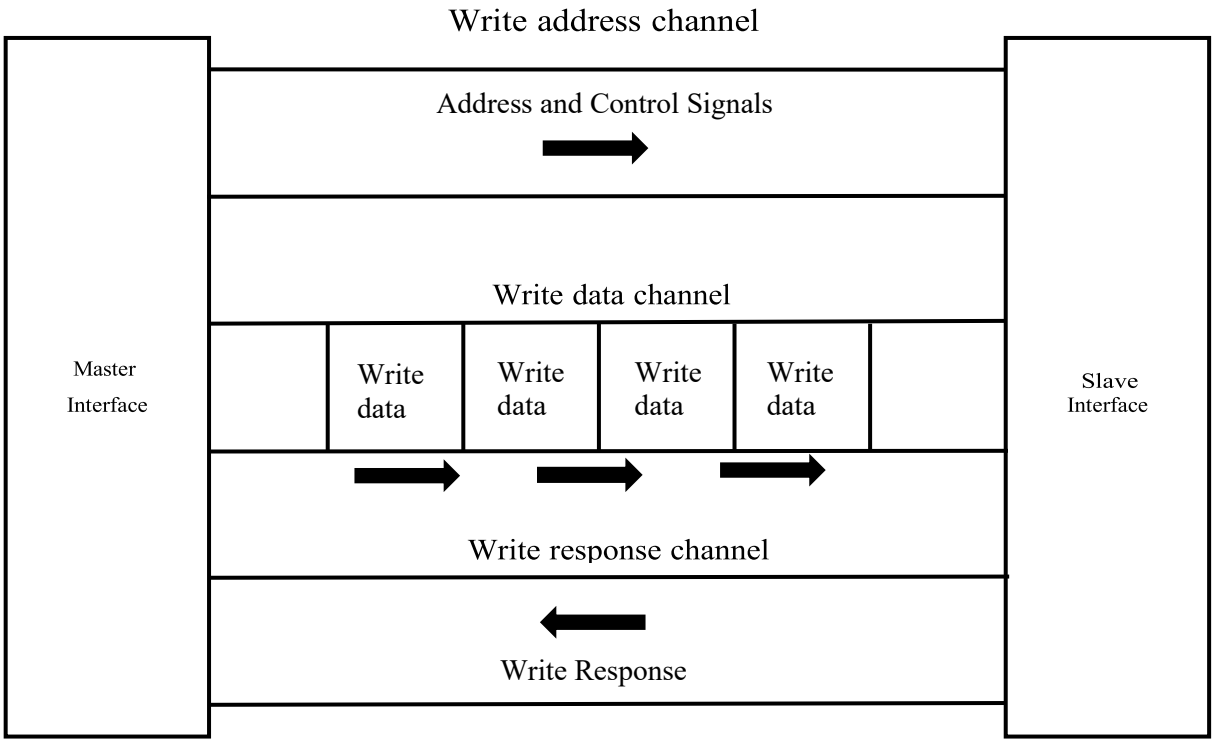


Fig-3.2 Write Transaction of AXI Interface

3.3.2 Read Operation in AXI

The AXI-Lite read operation is a simplified version of the AXI protocol read transaction, designed for low-complexity and low-throughput applications such as register access and control interfaces. Unlike full AXI, AXI-Lite does not support burst transfers and allows only single data transactions, which significantly reduces design complexity. The read operation in AXI-Lite uses two independent channels: the read address channel and the read data channel, both of which follow a handshake mechanism using VALID and READY signals.The read transaction begins when the AXI-Lite master places a valid read address on the read address channel by asserting ARADDR (read address) along with ARVALID (address valid signal). Since AXI-Lite does not support burst transfers, signals like ARLEN, ARSIZE, and ARBURST are not used. The slave monitors these signals and, when ready to accept the address, asserts ARREADY. The handshake between ARVALID and ARREADY completes the address phase of the read operation.

Once the address is accepted, the slave processes the read request and prepares the corresponding data. The data transfer occurs on the read data channel using signals such as RDATA (read data), RVALID (data valid signal), and RRESP (response signal indicating status such as OKAY or ERROR). The slave asserts RVALID when the data is ready, and the

master responds by asserting RREADY to accept the data. The transfer is completed when both RVALID and RREADY are high in the same clock cycle.Since AXI-Lite supports only single transfers, there is no concept of burst or multiple data beats, and therefore no need for signals like RLAST. This makes the protocol simpler and easier to implement compared to full AXI. The separation of address and data channels still allows some level of flexibility, but the overall operation remains sequential and straightforward.

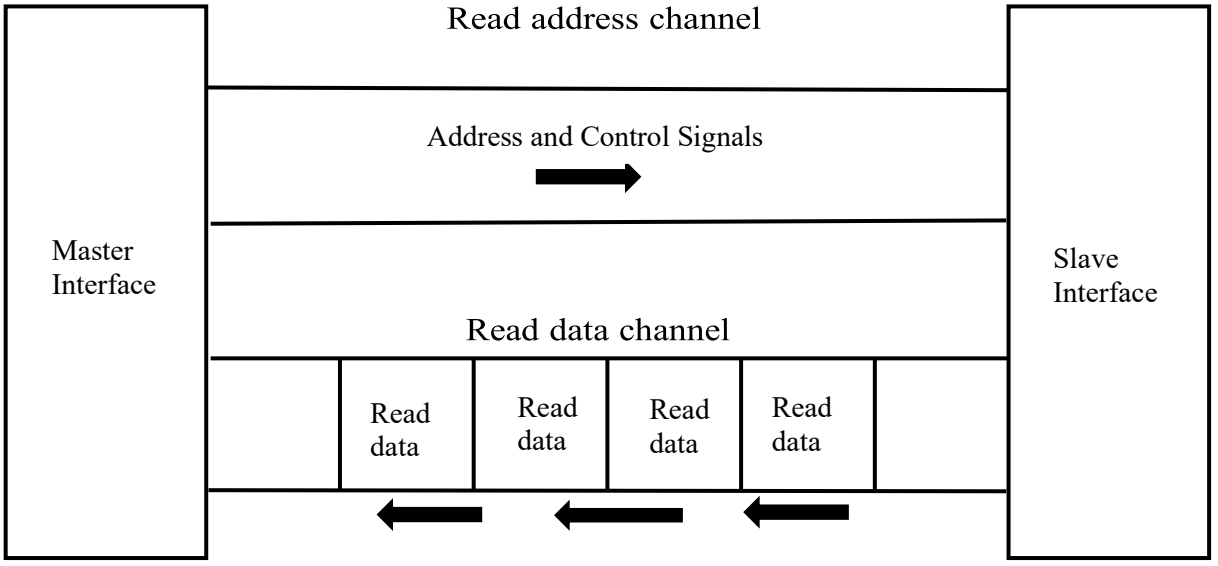


Fig 3.3- Read Transaction of AXI interface

3.4 Sigal Description of AXI-Lite Protocol

The simulation of AXI signals is a critical step in verifying the correct operation of the AXI Lite interface in the proposed bridge design. AXI protocol is based on a handshake mechanism that uses VALID and READY signals to ensure reliable data transfer between master and slave components. In this project, the AXI-Lite interface is simulated using a SystemVerilog testbench in Xilinx Vivado, and the resulting waveforms are analyzed to confirm correct protocol behavior and signal timing. At the beginning of the simulation, the system is initialized using a reset signal. During this phase, all AXI signals such as address, data, and control signals are set to their default inactive states. This initialization is important to ensure that the system starts from a known and stable condition, avoiding undefined behavior. The reset phase also allows internal registers and FSM states to be properly initialized before any transaction begins.

The AXI write operation begins when the master asserts the write address valid signal (AWVALID) along with a valid address on the AWADDR bus. This indicates that a write request is being initiated. Simultaneously or shortly after, the write data valid signal (WVALID) is asserted along with the corresponding data on the WDATA bus. These signals together represent a complete write transaction request from the AXI master. The bridge responds to the write request by asserting the ready signals AWREADY and WREADY. These signals indicate that the bridge is ready to accept the address and data provided by the master. The handshake is completed when both VALID and READY signals are high at the same clock edge. This handshake mechanism ensures proper synchronization and prevents data loss or corruption during transfer.

Once the write transaction is accepted, the bridge processes the data and generates a write response signal (BVALID). This signal indicates that the write operation has been successfully completed. The AXI master acknowledges this response by asserting BREADY, completing the write transaction cycle. The waveform clearly shows this sequence, confirming correct write operation behavior.For read operations, the AXI master initiates the transaction by asserting the read address valid signal (ARVALID) along with the address (ARADDR). This indicates that the master is requesting data from the specified address. The bridge responds by asserting ARREADY, completing the handshake for the read address channel.

The waveform also confirms that the bridge generates appropriate response signals for each transaction. For write operations, the BVALID signal is asserted only after the transaction is completed, and for read operations, the RVALID signal is asserted when valid data is available. This ensures proper communication and acknowledgment between the master and the bridge. The AXI signals remain stable during data transfer, with no glitches or unexpected transitions observed in the waveform. This stability is essential for reliable operation, especially in high-speed communication systems. It also indicates that the design is robust and well implemented. Another key aspect observed in the simulation is that the bridge correctly interprets AXI control signals and converts them into appropriate internal operations. The transition from AXI signals to APB signals is smooth and consistent, demonstrating effective protocol conversion.

3.5 Description of APB Protocol

The Advanced Peripheral Bus (APB) is a part of the AMBA protocol family developed by ARM for communication with low-speed peripherals in System-on-Chip (SoC) designs. It is mainly used for devices such as UART, timers, and GPIO, where high performance is not required. Compared to AXI and AHB, APB has a simple architecture with fewer signals, making it easy to design and power-efficient. Its non-pipelined nature allows only one transfer at a time, which reduces complexity and makes it suitable for control-based operations. The operation of the APB protocol is based on two phases: setup and enable. In the setup phase, the master selects the slave and provides address, control, and data signals while the enable signal remains low. In the next cycle, the enable phase begins where the enable signal is asserted, and the data transfer takes place. The slave responds using signals like ready and read data. Both read and write operations follow this simple sequence, ensuring reliable communication between master and slave. Although APB is simple and power-efficient, it has limitations such as lack of pipelining and lower speed compared to AXI and AHB protocols. Therefore, it is mainly used for low-bandwidth applications and is often connected to high-performance buses through bridge designs like AXI to APB. Overall, APB provides an efficient and reliable solution for peripheral communication in SoC systems.

The operation of the APB protocol is based on a simple two-phase transfer mechanism: the setup phase and the enable phase. During the setup phase, the master selects the target slave device by asserting the select signal and provides the necessary address, control signals, and write data if required. At this stage, the enable signal remains low, indicating that the transfer has not yet started. In the next clock cycle, the enable phase begins when the enable signal is asserted. During this phase, the actual data transfer occurs, and the slave processes the request. The slave then responds with appropriate signals such as ready (indicating completion of the transfer) and read data in case of a read operation. This two-step process ensures clear timing separation between address/control setup and data transfer, leading to reliable communication.

Both read and write operations in APB follow this same sequence, which makes the protocol predictable and easy to design. In a write operation, the master sends data to the slave during the enable phase, while in a read operation, the slave provides data back to the master. The use of simple handshaking signals ensures that data transfer is completed correctly without loss or corruption. Additionally, the APB protocol allows for wait states if the slave is not ready to complete the transfer immediately, further enhancing its reliability.

Despite its advantages, APB has certain limitations. Its lack of pipelining and inability to handle multiple transactions simultaneously result in lower performance compared to AXI and AHB protocols. It is not suitable for high-bandwidth applications or systems requiring rapid data transfers. However, these limitations are acceptable in scenarios where simplicity, low power consumption, and ease of integration are more important than speed. To overcome these limitations in complex systems, APB is typically connected to high-performance buses like AXI or AHB through bridge modules such as AXI to APB bridges. These bridges enable efficient communication between high-speed processors and low-speed peripherals, combining the strengths of both bus types. In conclusion, the APB protocol provides a simple, efficient, and reliable solution for connecting low-speed peripherals in SoC designs. Its minimalistic design reduces power consumption and hardware complexity, making it highly suitable for embedded applications. Although it does not support high-speed data transfer, its ease of implementation and dependable operation make it an essential component in modern bus architectures, particularly when used in conjunction with high-performance buses through bridge-based designs.

3.6 Signal Description of APB Protocol

All APB signals are initialized to their default inactive states. Signals such as PSEL, PENABLE, PWRITE, PADDR, and PWDATA remain low or undefined until a valid transaction is initiated from the AXI side. This idle condition ensures that no unintended operations occur on the APB interface and that the system remains stable until a valid request is received.

When a write request is generated from the AXI interface, the bridge initiates the APB transaction by entering the SETUP phase. In this phase, the PSEL signal is asserted high to select the peripheral, while PADDR, PWDATA, and PWRITE signals are driven onto the bus. The PENABLE signal remains low, indicating that the transfer is being prepared. In the next clock cycle, the system moves to the ACCESS phase where PENABLE is asserted high and PSEL remains high, showing that the transfer is active. During write operations, data from PWDATA is transferred to the peripheral, and PWRITE stays high throughout. The PREADY signal indicates completion of the transfer, and in this design, it is asserted immediately, meaning no wait states are required. For read operations, PWRITE is low, and the peripheral places data on PRDATA during the ACCESS phase, which is then sent back to the AXI interface.

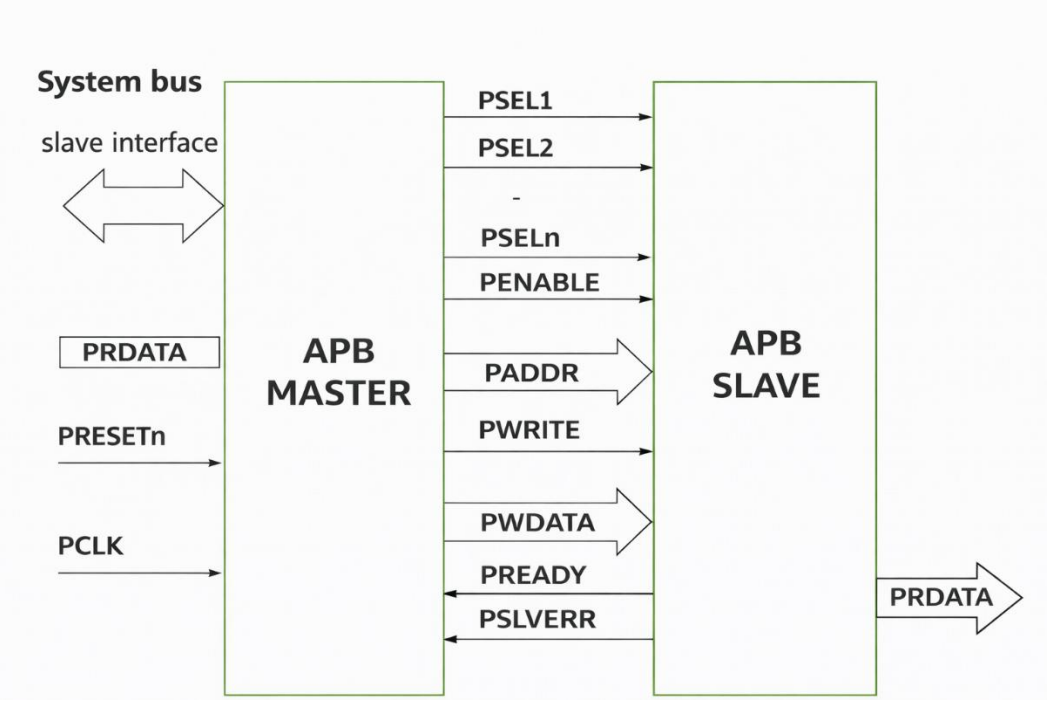


Fig-3.5 Block diagram of APB Signals

The simulation results clearly show that the APB protocol follows a strict two-phase operation, where every SETUP phase is followed by an ACCESS phase without overlap, confirming its non-pipelined nature. All signals such as address, data, and control remain stable during the ACCESS phase, ensuring reliable communication. The timing relationship between PSEL and PENABLE is also correctly maintained, with PSEL active in both phases and PENABLE active only in the ACCESS phase. The waveform confirms proper signal transitions and correct data transfer for both read and write operations. Overall, the results validate that the APB protocol is correctly implemented and the bridge successfully performs AXI to APB protocol conversion.

3.7 Description AXI T0 APB Bridge

The Finite State Machine (FSM) acts as the core control unit of the AXI to APB bridge and is responsible for controlling the overall working of protocol conversion. It manages the sequence of operations required to convert AXI transactions into APB transactions in a structured manner. The FSM ensures proper coordination between different signals and maintains correct timing throughout the process. By controlling the transitions between different states, it guarantees that data transfer occurs smoothly and without errors.

Initially, when the system is reset, the FSM enters the IDLE state, where it waits for a valid transaction request from the AXI interface. In this state, no operation is performed and all control signals remain inactive. When a valid read or write request is detected, the FSM moves to the SETUP state, which marks the beginning of the APB transaction. In this state, the required address, data, and control signals are prepared and applied to the APB interface. The PSEL signal is asserted to select the peripheral, while PENABLE remains low, indicating that the transfer is being prepared

In the next clock cycle, the FSM transitions to the ACCESS state, where the actual data transfer takes place. In this state, PENABLE is asserted high while PSEL remains active, indicating that the transfer is in progress. The FSM ensures that all signals remain stable during this phase so that the peripheral can correctly process the request. It also monitors the PREADY signal to determine whether the transaction is complete. Since PREADY is always high in this design, the transfer completes without any wait states, improving efficiency. After the completion of the transaction, the FSM returns to the IDLE state or proceeds to handle the next request. For write operations, the FSM keeps the PWRITE signal high, while for read operations it keeps it low, ensuring correct operation handling. The FSM strictly follows the sequence IDLE → SETUP → ACCESS → IDLE without skipping any states, which confirms reliable and error-free operation.Overall, the FSM provides a simple yet effective control mechanism for the AXI to APB bridge. It ensures correct signal timing, proper state transitions, and accurate protocol conversion. This structured working approach makes the design stable, efficient, and easy to implement in SoC systems.

3.8 Summary

The AXI to APB bridge design focuses on achieving efficient protocol conversion between a high-speed AXI interface and a low-speed APB interface using a structured and modular approach. The system is divided into key components such as AXI interface, FSM controller, and APB interface, where each module performs a specific function. The FSM plays a major role in controlling the sequence of operations by managing state transitions like IDLE, SETUP, and ACCESS to ensure proper data transfer.

The design maintains correct signal timing, data integrity, and synchronization between different stages of communication. Address decoding and controlled signal generation help in selecting the correct peripheral and executing transactions accurately. Both read and write operations are handled efficiently without data loss.Overall, the proposed design provides a simple, reliable, and effective solution for AXI to APB protocol conversion. It ensures smooth communication between different speed domains while maintaining low complexity.

CHAPTER – 4

SYSTEM IMPLEMENTATION

4.1 Introduction of AXI Lite-APB Bridge Implementation

The Implementation of the AXI to APB bridge is carried out using Verilog/SystemVerilog to achieve efficient protocol conversion between AXI and APB interfaces. The design follows a modular approach, where each functional block is implemented separately and then integrated to form the complete system. This approach improves readability, reusability, and ease of debugging. The implementation mainly focuses on correct signal handling, proper state transitions, and reliable data transfer between master and slave modules.

The design is implemented using hardware description languages such as Verilog and SystemVerilog. Verilog is used for describing the structural and behavioral aspects of the bridge, while SystemVerilog is used for enhanced verification and testbench development. The code includes different always blocks, combinational logic, and sequential logic to control signal flow and state transitions. The FSM is implemented using a case statement inside a clocked always block, where each state (IDLE, SETUP, ACCESS) is defined and transitions are controlled based on input conditions. Control signals like PSEL, PENABLE, and PWRITE are generated according to the current state of the FSM. Data paths are handled using registers and wires to ensure proper synchronization between AXI and APB domains. The code also includes proper reset conditions to initialize all signals and states. This ensures that the system starts in a known state and avoids unexpected behavior. Overall, the coding style follows a structured and modular design approach for better clarity and maintainability. The AXI to APB bridge is divided into multiple modules, each responsible for a specific function

4.2 Implementation Of AXI Lite Interface

First, the AXI-Lite interface is created using the interface keyword, where all required signals are declared in one place. The clock (ACLK) and reset (ARESETn) signals are defined as inputs to control the timing and initialization of the design. After that, all five AXI-Lite channels are declared: write address, write data, write response, read address, and read data. Each channel includes valid and ready signals, which are used for handshake communication. This step ensures that all required signals are available before connecting modules. Once the interface is defined, it can be instantiated inside both master and slave modules. This avoids

rewriting signals multiple times. It also keeps the design clean and structured. At this stage, no logic is implemented—only signal definition is done.

The execution of the AXI-Lite interface is completely based on the clock signal ACLK and happens in a synchronous manner. All operations are triggered on the rising edge of the clock. Initially, when the reset signal ARESETn is low, all signals are cleared and the system goes into an idle state. Once the reset is released (set to high), the interface becomes active and ready for transactions. At this stage, all valid signals (AWVALID, WVALID, ARVALID) are low, and no communication is happening. The master then initiates either a read or write transaction depending on the requirement. The entire execution is controlled using handshake signals, which ensures that data transfer happens only when both master and slave are ready. This prevents data loss and ensures proper synchronization between modules. Each transaction is divided into multiple phases, and every phase is completed only after a successful handshake.

In a write operation, execution starts when the master places a valid address on AWADDR and sets AWVALID high. This indicates that the address is ready to be transferred. The slave monitors this signal and, when ready, asserts AWREADY. Once both AWVALID and AWREADY are high at the same clock edge, the address transfer is completed. Next, the master sends the write data on WDATA and sets WVALID. The slave responds with WREADY when it is ready to accept the data. When both signals are high, the data transfer is completed. After receiving both address and data, the slave processes the write request internally. Then it generates a response using BRESP to indicate success or error and sets BVALID. The master acknowledges this by asserting BREADY. When both are high, the write transaction is fully completed and the system returns to idle or waits for the next operation.

In a read operation, execution begins when the master places the address on ARADDR and sets ARVALID high. The slave checks this request and responds by asserting ARREADY when it is ready. Once both signals are high, the address transfer is completed. The slave then processes the request and fetches the required data. After that, it places the data on RDATA and sets RVALID to indicate that valid data is available. It also sends a response signal RRESP to indicate the status of the transaction. The master waits for RVALID and then asserts RREADY to accept the data. When both RVALID and RREADY are high, the data transfer is successfully completed. After this, the system goes back to idle state or proceeds with the next transaction. This step-by-step handshake-based execution ensures reliable and efficient data transfer in AXI-Lite protocol.

4.3 Implementation of APB Interface

This interface includes all the standard APB signals required for communication between master and slave devices. The PADDR (Address) signal is used to specify the address for the transaction. The PSEL (select) signal selects the peripheral device for communication. The PENABLE (Enable) signal indicates the enable phase of the transfer. The PWRITE signal determines whether the operation is read or write. The PWDATA(Write) signal carries data from master to slave during write operations. The PRDATA (Read Data) signal carries data from slave to master during read operations. The PREADY signal indicates when the slave is ready to complete the transfer. The PSLVERR (Slave Error) signal is used to indicate any error during communication. These signals together ensure smooth and proper APB data transfer.

The interface is driven by the PCLK (Clock) signal, which controls the timing of operations. The PRESET (reset) signal is an active-low reset used to initialize the system. By using an interface, we can avoid repetitive signal declarations. This also improves readability of the code. Designers can easily understand how signals are connected. It supports better modular design practices. Overall, it makes the hardware design cleaner and easier to maintain. The APB protocol is designed for low-speed, low-power communication with peripheral devices such as UART, timers, and GPIO modules. Unlike high-performance protocols such as AXI or AHB, APB uses a simple, non-pipelined structure where only one transfer occurs at a time. This simplicity reduces hardware complexity and makes the protocol ideal for control and configuration tasks. The interface defined in the code includes all the essential signals required for APB communication between a master and a slave.

The PADDR signal is a 32-bit address line used to specify the location of the peripheral register that is being accessed. It is generated by the APB master and remains stable during the transfer. The PSEL signal is used to select a particular APB slave device. When this signal is asserted, it indicates that the slave is being targeted for a transaction. The PENABLE signal is used to indicate the second phase of the APB transfer, known as the enable phase. It is asserted after the setup phase and signals that the data transfer is taking place. The PWRITE signal determines the type of operation being performed. When PWRITE is high, it indicates a write operation, and when it is low, it represents a read operation. For write transactions, the PWDATA signal carries the data from the master to the slave. This is also a 32-bit signal and holds the data to be written into the selected peripheral register.

For read operations, the PRDATA signal is used to carry data from the slave back to the master. This signal contains the data read from the addressed location and is valid during the enable phase of the transfer. The PREADY signal is used by the slave to indicate whether it is ready to complete the transfer. If the slave requires additional time, it can de-assert PREADY to introduce wait states, ensuring reliable communication without data loss. The PSLVERR signal is used to indicate an error condition during a transfer. If an error occurs, such as an invalid address or failed operation, this signal is asserted by the slave to inform the master. This helps in error detection and handling within the system.

4.4 Implementation of FSM Interface

The module is responsible for controlling the communication between the AXI-Lite interface and the APB interface. It acts as the main controller that converts AXI requests into APB transactions using a Finite State Machine (FSM). The design uses a clock signal clk and an active-low reset control its operation. The FSM is implemented using three states: IDLE, SETUP, and ACCESS, which directly match the APB protocol behavior. The current state is stored in a state register and updated on every clock cycle. When reset is active, the FSM goes to the IDLE state.

The next state logic is written using a combinational block, which decides how the FSM transitions between states. This structured approach makes the design simple and easy to debug. The FSM ensures proper sequencing of signals between AXI and APB. Overall, it acts as the bridge control unit. Initially, the FSM stays in the IDLE state where no operation is performed. In this state, it continuously monitors the AXI side signals axi_wr_req (write Request) and axi_rd_req (Read Request). When either a read or write request is detected, the FSM moves to the SETUP state. In the SETUP state, the APB select signal PSEL is asserted. The PWRITE signal is set based on whether it is a read or write operation. After one clock cycle, the FSM automatically transitions to the ACCESS state.

In the ACCESS state, the PENABLE signal is asserted to start the actual data transfer. The FSM waits for the PREADY signal from the APB slave, which indicates that the transaction is complete. If PREADY is low, the FSM stays in the same state until the slave is ready. Once PREADY becomes high, the transfer is completed, and the done signal is asserted. For read operations, the data from PRDATA is directly assigned to axi_rdata. After completion, the FSM returns to the IDLE state and waits for the next request. This step-by-step execution ensures proper synchronization and reliable communication between AXI and APB protocols.

4.5 Implementation Of AXI-Lite TO APB Bridge Module

The AXI–APB Bridge acts as the top-level module that integrates all components of the design into a single functional unit. It connects the AXI interface, APB interface, and the Finite State Machine (FSM) to implement complete bridge functionality. This module plays a crucial role in coordinating the data flow between the high-speed AXI bus and the low-speed APB bus. It receives AXI transactions such as read and write requests, processes them through the FSM, and generates the corresponding APB control and data signals required for communication with peripheral devices. Additionally, it captures the responses from the APB side and forwards them back to the AXI master in the required format, ensuring protocol compliance.

The top-level module is responsible for maintaining proper synchronization and signal integrity between different components. It manages various control signals, including valid/ready handshakes on the AXI side and enable/select signals on the APB side. By coordinating these signals effectively, the module ensures smooth and reliable communication between the two interfaces. It also handles the conversion of AXI transactions into APB-compatible operations, taking into account differences such as burst support in AXI and single transfer operation in APB.Internally, the module organizes the interaction between submodules in a structured manner, ensuring that each block performs its designated function without conflict. The FSM within the design controls the sequence of operations, transitioning through different states such as idle, setup, and access, thereby ensuring correct timing and execution of transactions. The top module also ensures that data paths are correctly routed, allowing seamless transfer of address, control, and data signals between the interfaces.

Another important feature of the top-level module is its modular and scalable design. The clear separation of functionality into different submodules makes the system flexible and easy to modify. New features such as buffering mechanisms, error detection, or support for additional peripherals can be integrated without significantly altering the existing architecture. This modular approach improves maintainability and simplifies debugging and verification processes.Furthermore, the top-level module plays a key role during synthesis and implementation, as it defines how all submodules are interconnected within the overall system. It ensures that all signals are properly mapped and that there are no conflicts or mismatches between modules. This is essential for achieving correct functionality during simulation as well as successful deployment on hardware platforms such as FPGA.

CHAPTER – 5

RESULT ANALYSIS

5.1 RTL Analysis Of AXI-APB Bridge

RTL (Register Transfer Level) analysis is an important stage in digital design that provides a structural representation of the design before synthesis. It helps designers visualize how the written SystemVerilog code is interpreted in terms of registers, combinational logic, and interconnections. In this project, RTL analysis is performed using Xilinx Vivado to understand the internal architecture of the AXI-Lite to APB bridge.

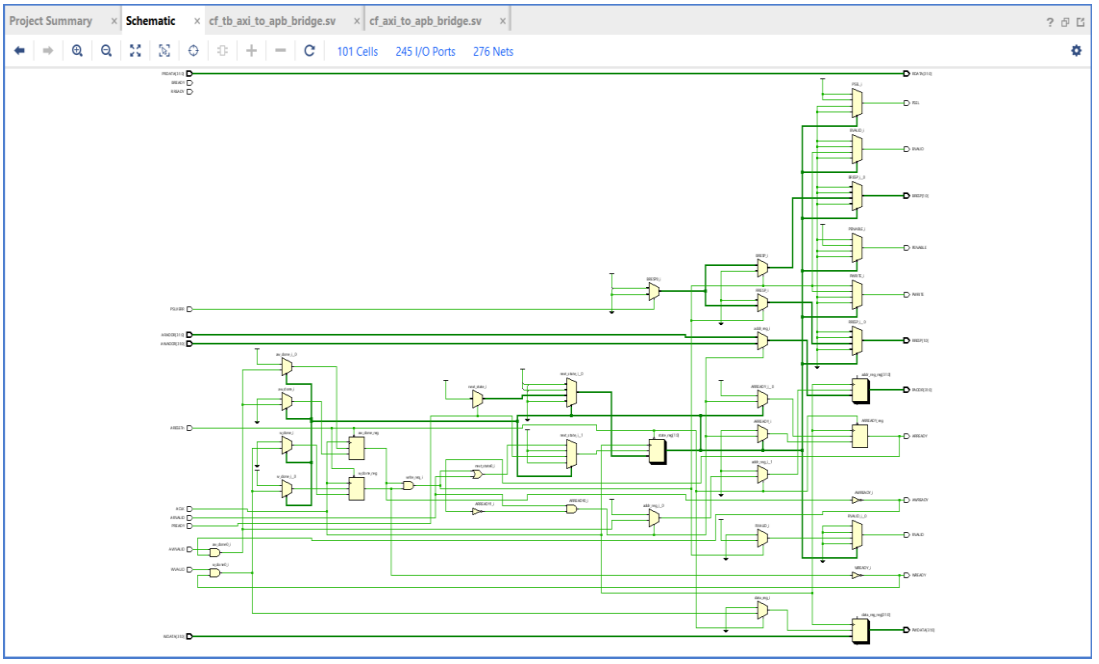


Fig 5.1-Schematic Diagram of AXI-APB Bridge

The RTL schematic (netlist diagram) generated by Vivado represents the logical structure of the design. It shows how different modules such as the AXI interface, APB interface, FSM, and top-level bridge module are interconnected. Each module is represented as a block, and the connections between them indicate signal flow. This graphical representation helps in verifying whether the design hierarchy is correctly implemented. One of the key observations from the RTL diagram is the modular organization of the design. The AXI interface module handles high-speed communication signals, while the APB interface module manages low-

speed peripheral signals.

The FSM module acts as the control unit, coordinating the operation between these two interfaces. The top module integrates all these components, ensuring proper communication and control flow.The RTL schematic also shows the presence of registers and combinational logic blocks. Registers are used to store data and maintain state information, while combinational logic is used to perform operations such as signal decoding and control generation. The correct placement of these elements ensures proper timing and synchronization within the design.

Another important aspect of RTL analysis is verifying signal connectivity. The diagram clearly shows how signals such as address, data, control, and handshake signals are routed between modules. This helps in identifying any missing connections or incorrect signal mappings that could lead to functional errors.The FSM logic is clearly visible in the RTL diagram, showing how different states are connected and how transitions occur based on input conditions. This provides a better understanding of the control flow and helps in verifying that the FSM is implemented correctly.

The RTL analysis also helps in identifying redundant logic or unnecessary complexity in the design. By examining the netlist, designers can optimize the design to reduce resource usage and improve performance. In this project, the RTL diagram shows a clean and efficient implementation without unnecessary components. Another observation is that the design follows a synchronous approach, where all operations are driven by a common clock signal. This ensures proper timing and avoids issues such as race conditions or glitches. The clock signal is distributed to all relevant modules, maintaining synchronization across the design.

The RTL schematic provides a clear view of how AXI transactions are converted into APB operations. It shows the flow of signals from the AXI interface through the FSM and into the APB interface, confirming correct protocol conversion at the structural level. Additionally, the RTL analysis confirms that the design is synthesizable. All constructs used in the SystemVerilog code are supported by synthesis tools, and there are no unsupported or ambiguous elements in the design

5.2 Synthesis Schematic Diagram

The synthesis schematic diagrams provide a gate-level representation of the design after the synthesis process. These diagrams are generated by Xilinx Vivado and illustrate how the high level SystemVerilog code is translated into hardware elements such as logic gates, flip-flops, multiplexers, and interconnections. This stage bridges the gap between RTL design and actual

hardware implementation. During synthesis, the Vivado tool analyzes the RTL code and maps it into an optimized netlist that can be implemented on the target FPGA. The schematic diagrams help designers visualize this transformation and verify whether the intended design structure is preserved after synthesis. These diagrams are essential for understanding how the logical design is physically realized.One of the key advantages of synthesis schematics is that they provide a detailed view of the internal architecture of the design. Unlike RTL diagrams, which show abstract representations, synthesis schematics display actual hardware components and their connections.

5.2.1 Schematic Diagram Of AXI-APB Bridge

The basic schematic diagram generated after synthesis provides a detailed gate-level representation of the overall design structure. It shows how the AXI-Lite to APB bridge is implemented using fundamental hardware elements such as logic gates, multiplexers, and flip flops. This diagram is essential for understanding how the high-level RTL description is translated into actual hardware components.In the basic schematic diagram, the entire design is represented as an interconnected network of modules and logic elements. The top-level module acts as the central block, connecting the AXI interface, APB interface, and FSM control logic. Each of these modules is further expanded into smaller components, revealing the internal implementation details.

One of the key observations in the schematic is the presence of combinational logic blocks used for signal decoding and control generation. These blocks are responsible for interpreting AXI control signals and generating corresponding APB signals. For example, signals such as PSEL, PENABLE, and PWRITE are derived from AXI inputs using combinational logic. Sequential elements such as flip-flops are also clearly visible in the diagram. These are used to store state information and maintain synchronization across clock cycles. The FSM is implemented using a set of flip-flops that hold the current state, along with combinational logic that determines the next state based on input conditions. The data path is another important aspect of the basic schematic diagram. It shows how data flows from the AXI interface to the APB interface during write operations and in the reverse direction during read operations. Multiplexers are used to control data selection and routing, ensuring that the correct data is transferred at the right time.

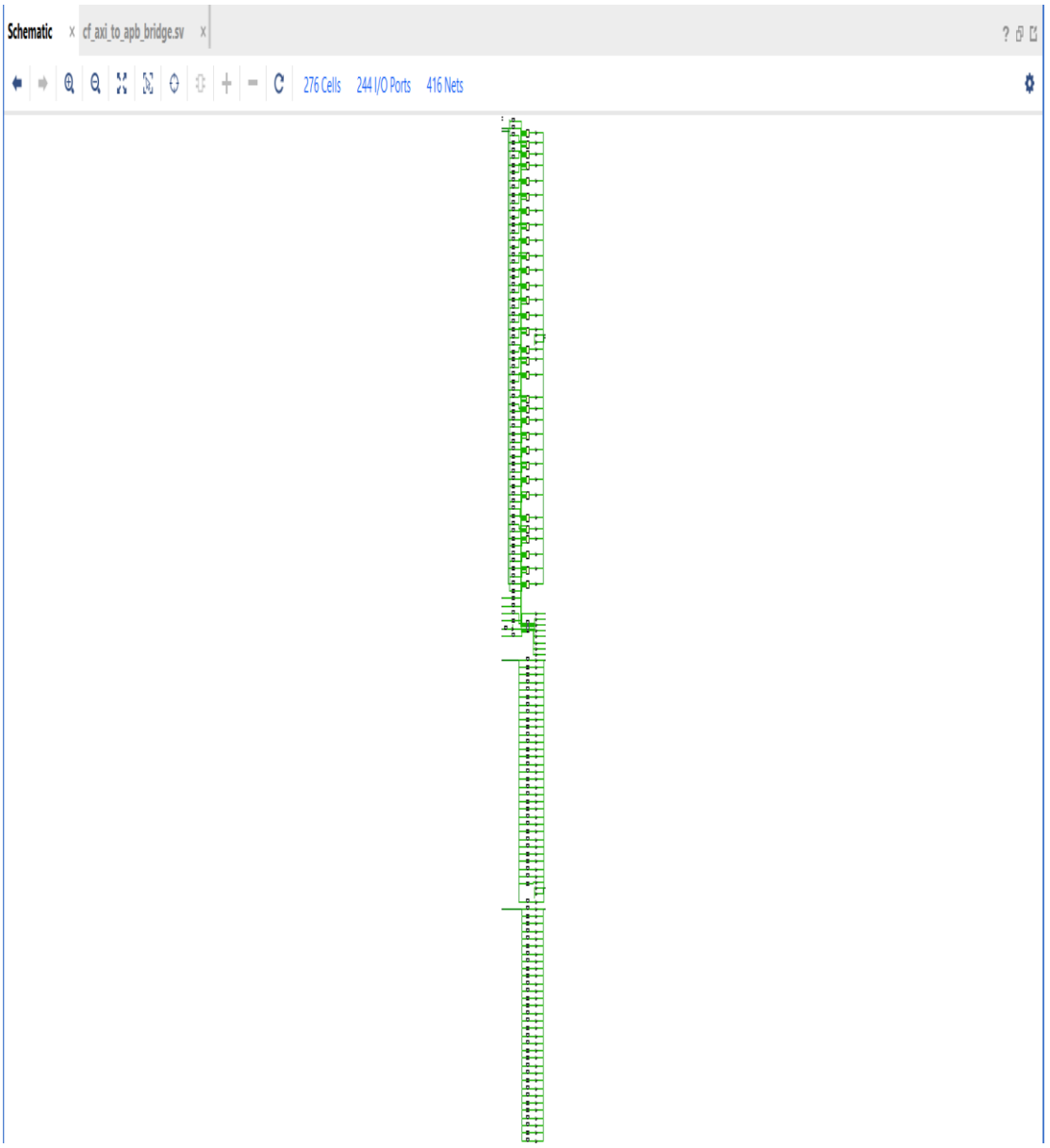


Fig5.2-Schematic Diagram of AXI-APB Bridge

Another important observation is the hierarchical organization of the design. The schematic maintains a clear hierarchy, where each module is represented as a separate block. This modular structure improves readability and makes it easier to analyze the design. The diagram also shows how the clock signal is distributed throughout the design. All sequential elements are driven by a common clock, ensuring synchronous operation. This is important for maintaining timing consistency and avoiding issues such as race conditions. In addition, the basic schematic diagram helps in identifying any redundant or unnecessary logic. The synthesis tool optimizes the design by removing unused signals and simplifying logic expressions. This results in a more efficient implementation with reduced resource usage.

The connectivity between different modules is clearly visible in the schematic. It shows how signals are routed from one module to another, ensuring proper communication and data flow. This helps in verifying that all connections are correctly implemented. Furthermore, the diagram helps in understanding the interaction between different components. It shows how the AXI interface communicates with the FSM, and how the FSM controls the APB interface. This provides a complete view of the system operation. Overall, the basic schematic diagram provides a comprehensive understanding of the hardware implementation of the AXI-to-APB bridge. It confirms that the design is well-structured, efficiently implemented, and ready for further stages such as placement and routing.

5.3 Power Analysis of AXI-APB Bridge

The power report generated by Xilinx Vivado provides an estimation of the total power consumption of the synthesized design. Power analysis is a crucial aspect of digital design, especially in modern System-on-Chip (SoC) systems where energy efficiency plays a significant role. The report gives detailed information about different components contributing to power consumption, such as dynamic power, static power, and I/O power. From the obtained power report of the AXI-to-APB bridge, it is observed that the overall power consumption of the design is relatively low. This is expected because the APB protocol is designed for low power peripheral communication, and the overall design is not highly complex. The efficient design of the FSM and limited number of logic elements contribute to reduced power usage.

The total on-chip power is composed of two main components: static power and dynamic power. Static power, also known as leakage power, is consumed even when the circuit is not switching. It depends on the FPGA technology and operating conditions. Dynamic power, on the other hand, is consumed due to switching activity in logic elements, interconnects, and I/O ports.In this project, the dynamic power is relatively low because the design does not involve high-frequency switching or complex computations. The AXI-Lite protocol used in the design supports single transactions rather than burst transfers, which reduces switching activity and hence lowers dynamic power consumption.

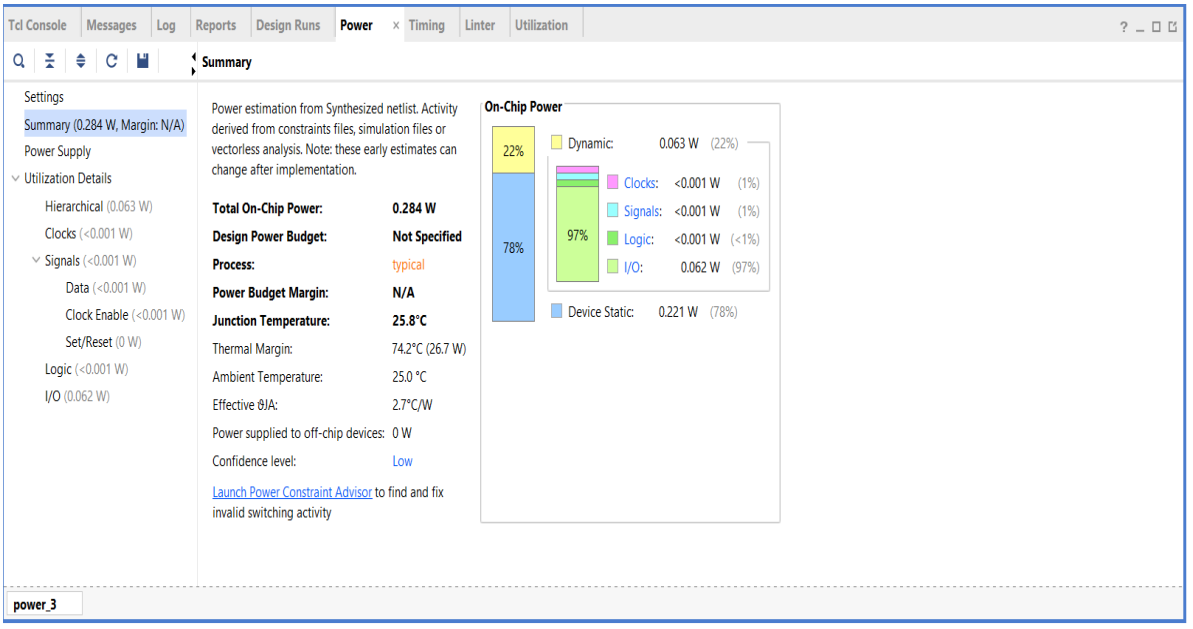


Fig5.3-Power Analysis of AXI-APB bridge

The clock network is one of the major contributors to dynamic power in most digital designs. However, in this design, the clock activity is moderate, and the number of sequential elements is limited. This results in reduced power consumption associated with the clock distribution network. Another component of power consumption is signal power, which arises due to switching in combinational logic and interconnects. The synthesis report indicates that signal power is minimal, reflecting the simplicity and efficiency of the design.

The I/O power is also a part of the total power consumption. It depends on the number of I/O ports and their switching activity. In this project, the number of I/O signals is moderate, and their activity is not very high, resulting in low I/O power consumption.The power report also provides information about the junction temperature of the FPGA device. The temperature remains within safe limits, indicating that the design does not cause excessive heating and is suitable for reliable operation.Another important observation is that there are no warnings related to excessive power consumption or thermal issues. This confirms that the design is energy-efficient and does not violate any power constraints.

The low power consumption of the design makes it suitable for embedded and portable applications, where energy efficiency is a critical requirement. The use of APB for peripheral communication further enhances power savings. The report also indicates that the design can operate efficiently at the specified clock frequency without significant power overhead. This ensures that performance is not compromised while maintaining low power consumption.

Additionally, the power analysis confirms that the design is scalable. Even if additional features are added in the future, the power consumption can be managed effectively through optimization techniques.

The use of SystemVerilog and structured design methodology also contributes to power efficiency by enabling optimized synthesis and reducing unnecessary logic. Another important aspect is that the design does not include any redundant switching or unused logic, which could otherwise increase power consumption. The synthesis tool has optimized the design to eliminate such inefficiencies. Overall, the power report confirms that the AXI-to-APB bridge is a low-power design, suitable for integration into modern SoC systems. The efficient use of resources

5.4 Utilization Report of AXI-APB Bridge

The utilization report generated by Xilinx Vivado provides detailed information about the hardware resources used by the design after synthesis. It helps in understanding how efficiently the FPGA resources are utilized and whether the design fits within the available device capacity. This report is essential for evaluating the scalability, efficiency, and feasibility of the design for real-world implementation. From the obtained utilization report of the AXI-to-APB bridge, it is observed that the design uses a relatively small number of FPGA resources. This indicates that the design is lightweight and optimized, making it suitable for implementation even on low-cost FPGA devices.

The minimal resource usage also allows room for integrating additional modules or functionalities in future expansions. The primary resources analyzed in the report include Look-Up Tables (LUTs), Flip-Flops (Registers), Block RAM (BRAM), and Digital Signal Processing (DSP) units. LUTs are used to implement combinational logic, while flip-flops are used for storing sequential data such as state information in the FSM.The report shows that the number of LUTs used in the design is low. This is because the design primarily consists of simple control logic and does not involve complex arithmetic operations. The efficient use of combinational logic ensures that unnecessary logic is minimized, reducing resource consumption.

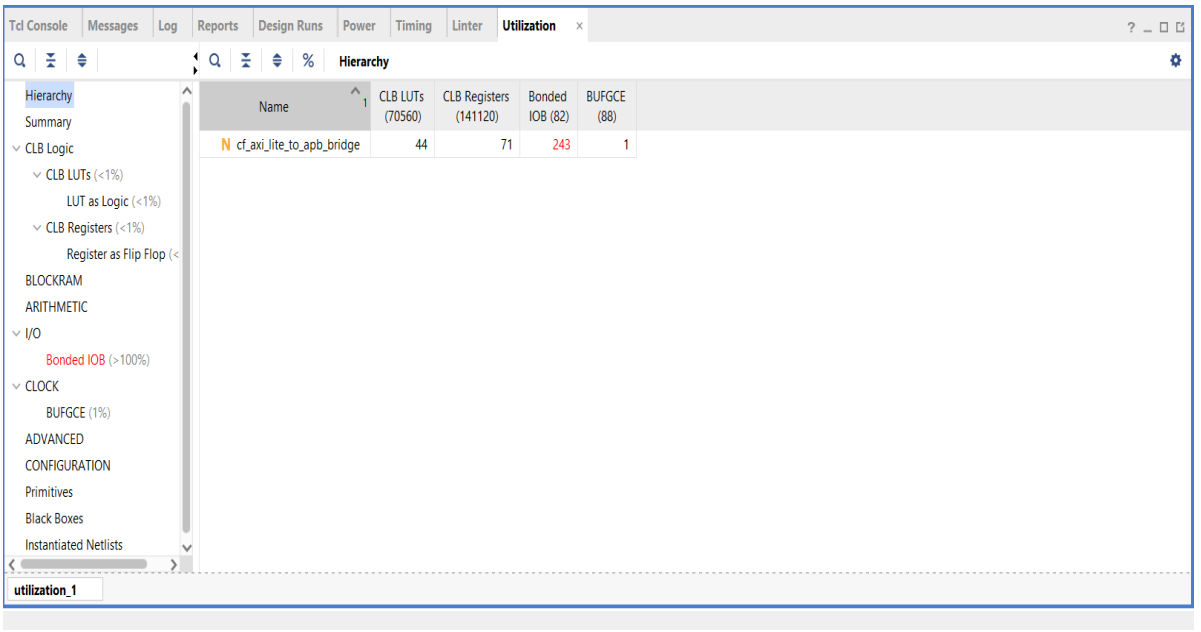


Fig 5.4 Utilization report of AXI-APB Bridge

Similarly, the number of flip-flops used is also limited. These registers are mainly used in the FSM to store the current state and in some parts of the data path. The low number of flip-flops indicates that the design is not heavily dependent on sequential storage, which contributes to its simplicity. Another important observation is that no Block RAM (BRAM) resources are used in the design. This is expected, as the AXI-to-APB bridge does not require large memory storage. The absence of BRAM usage further confirms that the design is lightweight and efficient. The report also indicates that no DSP slices are utilized. DSP units are typically used for complex arithmetic operations such as multiplication and filtering. Since this design focuses on protocol conversion and control logic, DSP resources are not required.

The utilization of I/O ports is also analyzed in the report. The number of I/O pins used corresponds to the AXI and APB interface signals. The report confirms that all required I/O ports are correctly defined and utilized without exceeding device limits. Another key aspectis the percentage utilization of resources. The report shows that only a small percentage of the available FPGA resources is used. This indicates that the design is highly efficient and leaves significant room for future enhancements. The hierarchical utilization breakdown provides insights into how different modules contribute to resource usage. It shows that the FSM and control logic consume a small portion of resources, while the interface modules contribute minimally. The report also confirms that there are no resource conflicts or over-utilization issues. All resources are used within safe limits, ensuring that the design can be successfully implemented on the selected FPGA device.

The efficient resource utilization also contributes to improved performance and reduced power consumption. Since fewer resources are used, the design experiences less routing complexity and lower switching activity. Another important observation is that the synthesis tool has optimized the design by removing redundant logic and simplifying expressions. This optimization results in reduced resource usage and improved efficiency. The utilization report also indicates that the design is scalable. Additional features such as buffering, error handling, or support for multiple peripherals can be added without significantly increasing resource usage. Furthermore, the low utilization makes the design suitable for integration into larger systems, where multiple modules share FPGA resources. This flexibility is important for SoC-based applications. Overall, the utilization report confirms that the AXI-to-APB bridge is efficiently designed, with minimal resource usage and high scalability. The design meets all resource constraints and is well-suited for FPGA implementation.

5.5 Timing Analysis Report of AXI Lite- APB Bridge

The timing report generated by Xilinx Vivado provides a detailed analysis of the timing performance of the synthesized design. It ensures that all signal paths in the design meet the required timing constraints, such as setup time, hold time, and clock period. Timing analysis is crucial in digital design because even a functionally correct design can fail if timing requirements are not satisfied. In this project, the timing report is analyzed after synthesis to verify whether the AXI-to-APB bridge operates correctly at the specified clock frequency. The report includes important parameters such as Worst Negative Slack (WNS), Total Negative Slack (TNS), and the maximum operating frequency of the design.

One of the key observations from the timing report is that the Worst Negative Slack (WNS) is positive. This indicates that all timing paths in the design meet the required setup time constraints. A positive slack means that signals arrive before the required time, ensuring reliable operation. Similarly, the Total Negative Slack (TNS) is zero, which confirms that there are no timing violations in the design. This indicates that all paths satisfy timing requirements and the design is free from setup time issues. The timing report also provides information about the critical path, which is the longest delay path in the design. In this project, the critical path is relatively short due to the simplicity of the design and the limited number of logic levels. This contributes to better timing performance.

Another important aspect is the clock frequency at which the design operates. The timing report confirms that the design can operate at the specified clock frequency without

violating timing constraints. This ensures that the design is stable and reliable under the given operating conditions. The setup time analysis ensures that data is stable before the clock edge, while the hold time analysis ensures that data remains stable after the clock edge. The report confirms that both setup and hold time requirements are satisfied in the design.

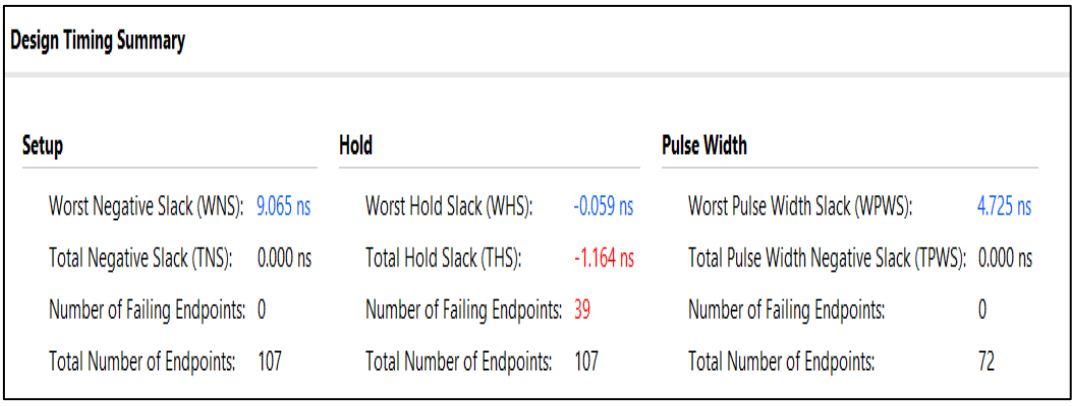


Fig-5.5 Timing Analysis of AXI Lite – APB Bridge

The timing report also includes information about clock skew and jitter. In this design, clock skew is minimal due to the use of dedicated clock routing resources in the FPGA. This ensures consistent timing across all sequential elements. Another observation is that the routing delays are minimal, indicating efficient placement and routing of the design. This contributes to improved timing performance and reduced propagation delays. The timing analysis also confirms that there are no combinational loops or unstable paths in the design. All paths are properly defined and synchronized with the clock signal.

The FSM plays an important role in timing performance, as it controls the sequence of operations. The timing report confirms that state transitions occur within the required time limits, ensuring correct operation. The report also shows that input and output delays are within acceptable limits. This ensures proper communication between the design and external components. Another important aspect is that the design does not include long combinational paths, which could lead to timing violations. The use of structured design techniques helps in maintaining short and efficient paths. The timing report also confirms that the design is scalable. Even if additional features are added, timing constraints can still be met with proper optimization. The absence of timing violations indicates that the design is ready for hardware implementation. It ensures that the AXI-to-APB bridge will function correctly when deployed on an FPGA.Overall, the timing report confirms that the design meets all timing requirements and operates reliably at the specified clock frequency. This validates the correctness and robustness of the design from a timing perspective.

5.7 Simulation Results

5.7.1Simulation Results of AXI Interface

Simulation is an essential step in digital design to verify the functional correctness of a system before synthesis and hardware implementation. In this project, simulation is performed using Xilinx Vivado to validate the behaviour of the AXI-Lite to APB bridge designed in SystemVerilog. The main objective is to ensure accurate protocol conversion between AXI and APB, along with correct operation of control signals, data transfers, and state transitions. Through simulation, the interaction between different modules such as the AXI interface, APB interface, and FSM is verified to ensure that the bridge performs as expected under various operating conditions.

SystemVerilog testbench is used to generate AXI read and write transactions as input stimuli, simulating real communication between a processor and peripheral devices. Various input conditions such as addresses, data values, control signals, and reset scenarios are applied to thoroughly test the design. The testbench drives signals like ARADDR (read address), AWADDR (write address), WDATA (write data), and control signals such as VALID and READY to mimic actual AXI transactions. Different address locations are used to verify whether the bridge correctly decodes and maps them to the appropriate APB slave. Similarly, different data values are applied to ensure accurate data transfer from AXI to APB and vice versa.

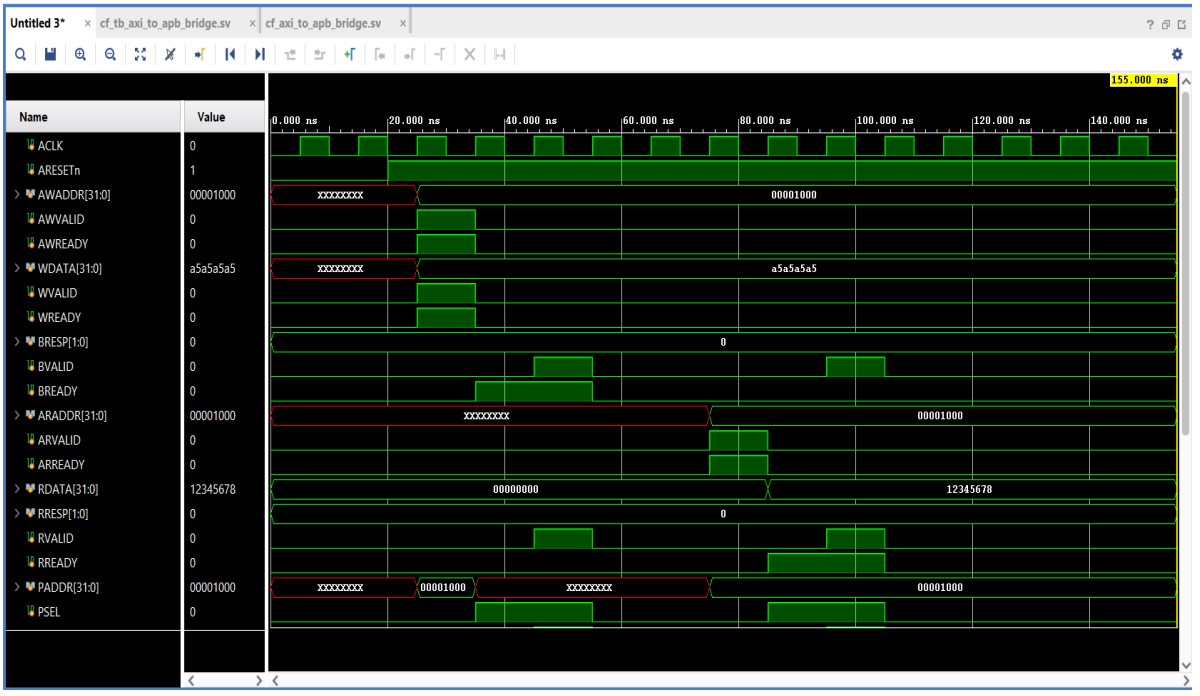


Fig-5.6 Simulation Results of AXI Interface

The given timing waveform represents the simulation results of the AXI-Lite to APB bridge, demonstrating both read and write operations along with the associated control and data signals. The waveform verifies the correct functionality of the bridge by showing proper interaction between the AXI and APB protocols. The system operates based on the clock signal (ACLK), and initially, the reset signal (ARESETn) is asserted to initialize the design. Once the reset is , the system begins normal operation, and transactions are initiated.

During the write operation, the AXI master provides the write address and data through signals such as AWADDR and WDATA, along with control signals AWVALID and WVALID. In the waveform, the address value **00001000** and data value **a5a5a5a5** can be observed. When the slave is ready, it asserts AWREADY and WREADY, completing the handshake process. After successful address and data acceptance, the bridge converts these AXI signals into corresponding APB signals. The APB address (PADDR) is generated as **00001000**, and the write data is transferred as **a5a5a5a5**. The PSEL signal is asserted to select the APB slave, and the write operation is completed successfully. A response is then sent back to the AXI master using BVALID and BRESP signals, indicating a successful transaction.

For the read operation, the AXI master initiates the transaction by placing a valid address on ARADDR along with ARVALID. In the waveform, the address **00001000** is used for the read operation. The slave responds by asserting ARREADY, indicating that the address has been accepted. The bridge then performs the corresponding APB read operation by forwarding the address to the APB interface. After processing, the APB slave returns the requested data, which is observed as **12345678** on the RDATA signal. The RVALID signal indicates that the data is valid, and the AXI master asserts RREADY to complete the data transfer The waveform clearly illustrates the VALID–READY handshake mechanism used in AXI protocol, where data transfer occurs only when both VALID and READY signals are asserted. This ensures proper synchronization and reliable communication between the master and slave. Additionally, the waveform reflects the internal operation of the bridge, where the Finite State Machine (FSM) controls the transition between different states such as idle, setup, and access during APB transactions.

5.7.2 Simulation Results of APB Interface

The simulation of APB signals is used to verify the correct operation of the low-speed peripheral interface in the AXI-to-APB bridge. APB follows a simple two-phase mechanism,

namely SETUP and ACCESS, which is clearly observed in the waveform. Initially, all APB signals such as PSEL, PENABLE, and PWRITE remain in an inactive state, ensuring stable operation until a valid transaction is initiated from the AXI side. This idle condition confirms that the bridge does not perform any unintended operations before receiving a request. When a valid write transaction is received, the bridge enters the SETUP phase, where PSEL is asserted high to select the APB slave. At this stage, the address and data signals become valid.

From the waveform, the address PADDR = 00001000 can be observed, which corresponds to the address provided by the AXI interface. Similarly, the write data PWDATA = a5a5a5a5 is driven onto the APB bus. During this phase, PWRITE is asserted, indicating a write operation, while PENABLE remains low, as per the APB protocol specification. In the next clock cycle, the bridge transitions into the ACCESS phase, where PENABLE is asserted high along with PSEL. This indicates that the data transfer is actively taking place between the bridge and the APB slave. The address and data signals remain stable during this phase to ensure correct data transfer. The waveform clearly shows that both PADDR and PWDATA maintain their values (00001000 and a5a5a5a5) throughout the ACCESS phase, which is a key requirement of the APB protocol.

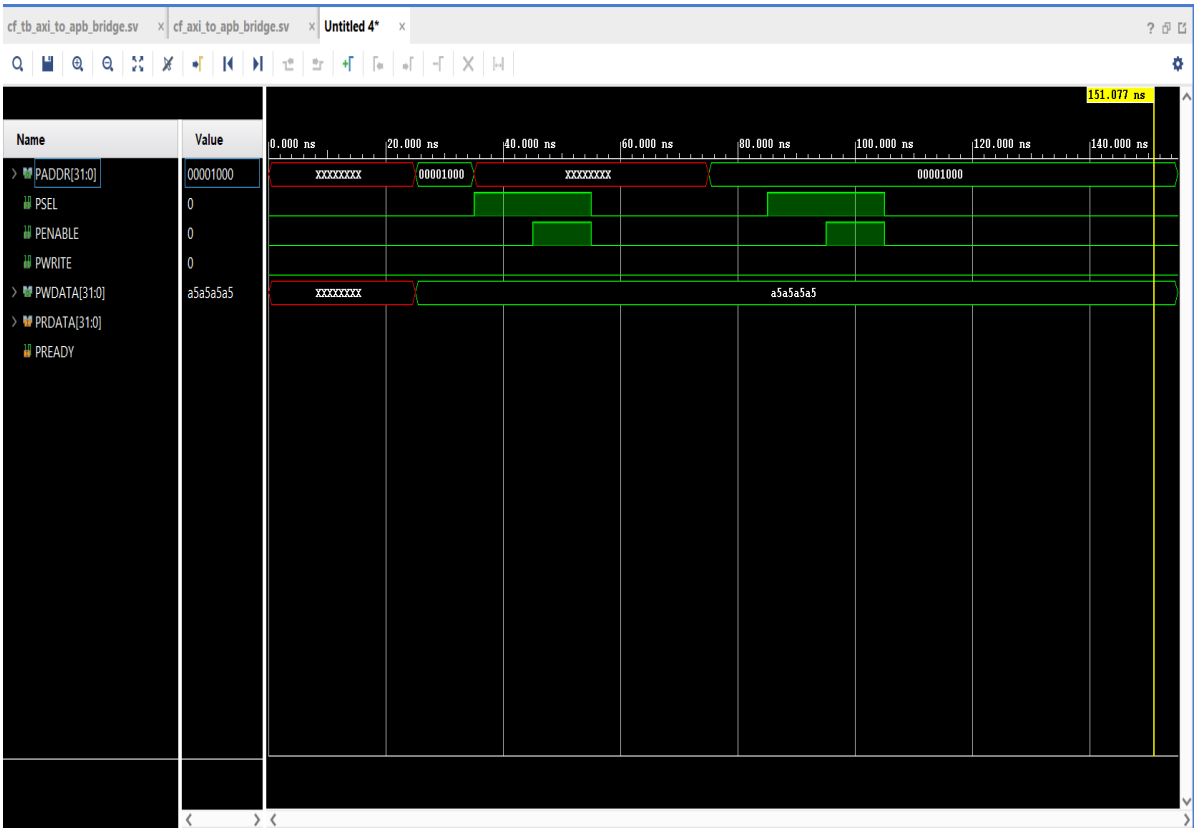


Fig 5.8 Simulation Results of APB Interface

The PREADY signal is used to indicate the completion of the transfer. In this design, PREADY is asserted without delay, meaning that the APB slave is always ready to complete the transaction in a single cycle. This avoids wait states and simplifies the timing behavior. Once PREADY is asserted, the transfer is considered complete, and the control signals return to their idle state in the following cycle. The waveform also demonstrates proper sequencing between multiple transactions. After completing one transfer, the signals return to idle before the next SETUP phase begins. This ensures that each APB transaction is clearly separated and follows the correct protocol timing. The transitions of PSEL and PENABLE clearly indicate the boundaries between SETUP and ACCESS phases, making it easy to verify correct FSM operation.

Overall, the timing waveform confirms that the APB interface is functioning correctly, with proper assertion and de assertion of control signals, stable address and data values during transfer, and correct synchronization between phases. The accurate representation of address (00001000) and data (a5a5a5a5) in the waveform validates that the AXI to APB bridge successfully converts and transfers transactions without any data loss or protocol violation.

5.7.3 Simulation Results of FSM Interface

The given waveform represents the operation of the Finite State Machine (FSM) used in the AXI to APB bridge, illustrating the transitions between different states such as IDLE, SETUP, and ACCESS. The waveform confirms that the FSM transitions occur smoothly and at appropriate clock edges, ensuring proper synchronization of all control and data signals. Initially, the FSM remains in the IDLE state, where no transaction is active, and control signals such as PSEL and PENABLE are deasserted, indicating that the APB interface is inactive. When a valid request is received from the AXI side, indicated by signals such as axi_wr_req or axi_rd_req, the FSM transitions from IDLE to the SETUP state. In this phase, the bridge prepares the APB signals for the transaction. The waveform shows that PSEL is asserted high, selecting the APB slave, while PENABLE remains low, as required by the APB protocol. At the same time, the address PADDR = 00001000 becomes valid, and for write operations, the data PWDATA = a5a5a5a5 is also driven onto the bus. This confirms that the FSM correctly captures and forwards the AXI address and data to the APB interface during the SETUP phase.

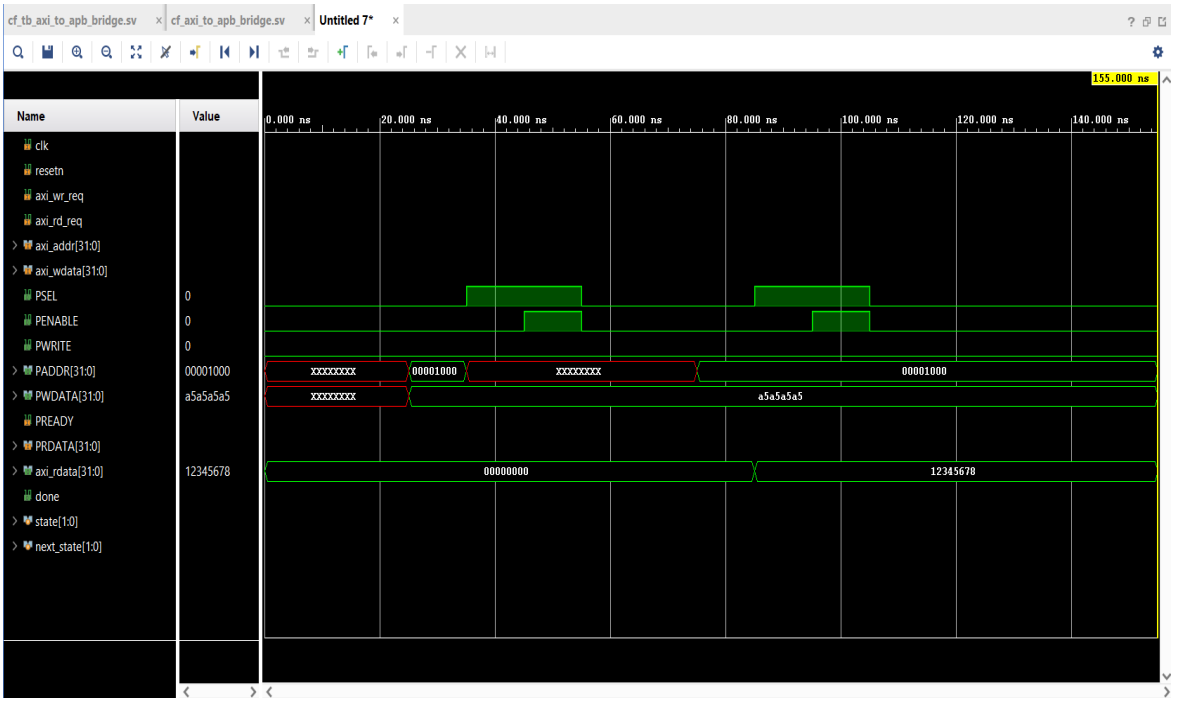


Fig-5.9 Simulation Results of FSM Interface

In the next clock cycle, the FSM moves to the ACCESS state, where PENABLE is asserted high along with PSEL, indicating that the actual data transfer is taking place. During this phase, the address and data signals remain stable, ensuring reliable communication. The waveform shows that PWRITE is asserted high during write operations, confirming that the FSM correctly identifies and executes write transactions. For read operations, PWRITE remains low, and the bridge receives data from the APB slave. This is evident from the waveform where PRDATA = 12345678 is observed, which is then forwarded to the AXI side as axi_rdata = 12345678.

The PREADY signal indicates the completion of the APB transfer. In this simulation, PREADY is asserted without delay, meaning the transaction completes in a single cycle without any wait states. Once the transfer is complete, the FSM transitions back to the IDLE state, resetting control signals such as PSEL and PENABLE. The waveform clearly shows that after completing one transaction, the FSM returns to IDLE before starting the next operation, ensuring proper sequencing and avoiding overlapping transactions.

The transitions strictly follow the sequence IDLE, SETUP, ACCESS and again IDLE, which confirms correct FSM design and implementation. Additionally, the signals state[1:0] and next_state[1:0] reflect the internal operation of the FSM, showing how the next state is determined based on current inputs and conditions. Overall, the waveform validates that the

FSM is functioning correctly, managing both read and write operations efficiently while maintaining proper protocol timing. The accurate generation of control signals, correct handling of address and data values (00001000, a5a5a5a5, 12345678), and proper state transitions confirm the reliability and correctness of the AXI to APB bridge design.

5.7.4 Simulation Results of AXI-APB Bridge

The given waveform represents the complete operation of the AXI-Lite to APB bridge, combining AXI interface signals, APB interface signals, and FSM control behaviour into a single view. This integrated waveform verifies the overall functionality of the bridge by showing how AXI transactions are successfully converted into APB operations under the control of the FSM. Initially, the system is in reset condition with ARESETn asserted, and all control signals remain inactive, indicating a stable starting state. Once the reset is deasserted, the system begins normal operation with the clock signal (ACLK) driving all transitions. During the write operation, the AXI master initiates the transaction by providing the write address AWADDR = 00001000 and write data WDATA = a5a5a5a5, along with control signals AWVALID and WVALID. The slave responds with AWREADY and WREADY, completing the handshake.

These values are then forwarded to the APB interface, where the FSM transitions from IDLE to SETUP state. In this phase, PSEL is asserted, and the corresponding APB address PADDR = 00001000 and data PWDATA = a5a5a5a5 are driven onto the bus. In the next clock cycle, the FSM enters the ACCESS state, where PENABLE is asserted, indicating active data transfer. After successful completion, the AXI side receives a response through BVALID and BRESP, confirming that the write operation is completed correctly. For the read operation, the AXI master issues a read request by asserting ARADDR = 00001000 and ARVALID. The slave acknowledges this with ARREADY, completing the address handshake. The FSM then controls the APB read operation by asserting PSEL and providing the address to the APB interface. During the ACCESS phase, the APB slave returns the data, which is observed as PRDATA = 12345678. This data is then forwarded back to the AXI interface

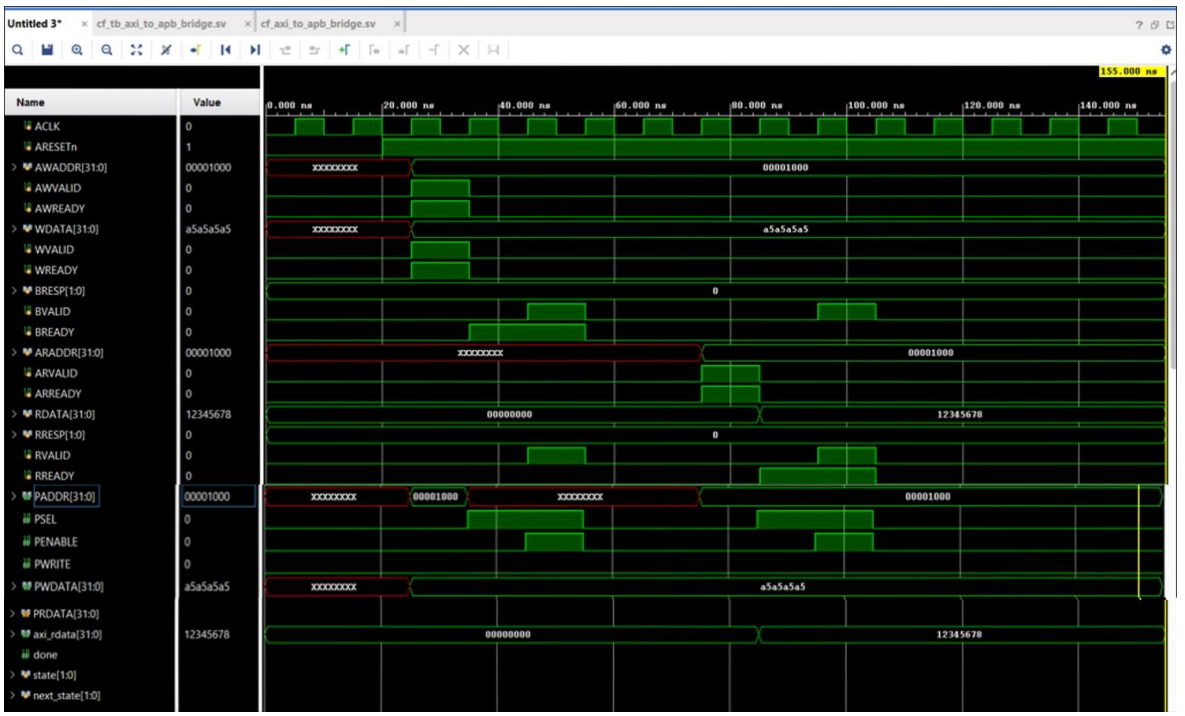


Fig 5.10 AXI-APB Bridge Waveform

The master completes the transaction by asserting RREADY, ensuring successful data transfer. The waveform also clearly shows the operation of the FSM through signals such as state and next_state, which follow the sequence IDLE, SETUP, ACCESS and IDLE for each transaction. The transitions occur at proper clock edges without any glitches or invalid states, confirming correct FSM implementation. Additionally, the VALID–READY handshake mechanism on the AXI side and the two-phase protocol (SETUP and ACCESS) on the APB side are clearly visible, demonstrating proper synchronization and protocol adherence.

CHAPTER – 6

CONCLUSION AND FUTURE SCOPE

6.1 Conclusion

This project successfully presents the design and implementation of an AXI-Lite to APB bridge using SystemVerilog, enabling communication between high-speed AXI-Lite and low-speed APB peripherals. The design effectively performs protocol conversion by translating AXI transactions into corresponding APB operations while maintaining data integrity and synchronization. A Finite State Machine (FSM) is used as the core control unit to manage transitions between IDLE, SETUP, and ACCESS states, ensuring proper sequencing and timing of operations. The AXI-Lite interface is implemented using VALID–READY handshake signals for reliable communication, while the APB interface follows a simple two-phase protocol, demonstrating efficient integration of both bus systems.

Simulation results using Xilinx Vivado confirm the functional correctness of the design, with waveform analysis showing proper signal transitions, accurate data transfer, and correct FSM behavior. Synthesis reports indicate that the design is lightweight, with minimal resource utilization and low power consumption, while timing analysis confirms stable operation under defined constraints. The modular architecture improves scalability and maintainability, making it suitable for future enhancements. Overall, the project demonstrates key concepts such as protocol conversion and FSM-based control, and highlights the importance of simulation and synthesis in digital design. The AXI to APB bridge is highly relevant in modern SoC architectures, and with further improvements, it can be extended to support more advanced and industry-level applications.

6.2 Future Scope

The AXI-Lite to APB bridge developed in this project provides a basic solution for protocol conversion in SoC architectures, but there is significant scope for further enhancement to improve performance, scalability, and reliability. One major improvement is extending the design to support the full AXI protocol, which includes burst transfers, multiple outstanding transactions, and out-of-order execution. This would greatly enhance system throughput and make the bridge suitable for high-performance applications. Additionally,

incorporating advanced error handling mechanisms such as SLVERR and DECERR can improve system reliability by detecting invalid accesses and communication faults. Performance can also be optimized using techniques like pipelining, buffering, and parallel processing to reduce latency and increase efficiency.

Further enhancements include supporting multiple APB peripherals through address decoding, making the design more practical for real-world applications. Parameterization of data width, address width, and peripheral count can improve flexibility and reusability. Verification can be strengthened using advanced methodologies like UVM for better coverage and validation. Power optimization techniques such as clock gating and dynamic voltage scaling can be applied to reduce power consumption. Moreover, implementing the design on FPGA hardware and integrating it into a complete SoC system can provide real-time validation. Additional features like security, protocol monitoring, and debugging interfaces can further enhance the design, making it more robust and suitable for modern embedded systems.

CHAPTER 7
REFERENCES

[1] N. Deshpande and R. Sadakale, et al. “AMBA AHB to APB Bridge Protocol Verification Using System Verilog,” in Proc. 2023 First Int. Conf. Advances in Electrical, Electronics and Computational Intelligence (ICAEECI)

[2] S. Saravanan, K. G. Devika, et al.“Verification of AMBA APB Bus Protocol Using UVM,” in Proc. 2025 3rd Int. Conf. Artificial Intelligence and Machine Learning Applications (AIMLA), Namakkal, India, 2025

[3] N. Gaikwad et al. “Verification of AMBA AXI On-Chip Communication Protocol,” in Proc. 2018 4th Int. Conf. Computing Communication Control and Automation (ICCUBEA), Pune, India, 2018

[4] C. Ma, Z. Liu, et al.“Design and implementation of APB bridge based on AMBA 4.0,” in Proc. 2011 Int. Conf. Consumer Electronics, Communications and Networks (CECNet), Xianning, China, 2011

[5]J. Mukunthan, A. Daniel Raj, et.al “Design and Implementation of AMBA APB Protocol,” Bannari Amman Institute of Technology, Sathyamangalam.

[6]G. Pan, L. Luo, et.al “Design and Verification of a MAC Controller Based on AXI Bus,” in Proc. 3rd Int. Conf. Intelligent System Design and Engineering Applications (ISDEA), 2013

[7]V. H. Vinay and B. S. Balaji, et.al “Design of Reusable VIP (Verification IP) of AXI Master and Slave in UVM,” in Proc. 2024 Int. Conf. Computing, Semiconductor, Mechatronics, Intelligent Systems and Communications (COSMIC), 2024.

[8]A. Gollapudi, A. Krishna Kumar, et.al “Implementation of an Optimized AXI using Verilog,” in Proc. 2024 7th Int. Conf. Circuit Power and Computing Technologies (ICCPCT), 2024

[9] M. Prasanna Deepu and R. Dhanabal, “Validation of Transactions in AXI Protocol using System Verilog,” in Proc. 2017 Int. Conf. Microelectronic Devices, Circuits and Systems (ICMDCS), 2017.

[10]K. Rawat, K. Sahni, et.al “Design of AMBA APB Bridge with Reset Controller for Efficient Power Consumption,” in Proc. 2014 9th Int. Conf. Industrial and Information Systems (ICIIS), 2014.

[11] A. Paunikar, R. Gavankar, et.al “Design and Implementation of Area Efficient, Low Power AMBA-APB Bridge for SoC,” in Proc. 2014 Int. Conf. Green Computing Communication and Electrical Engineering (ICGCCEE), 2014.

[12] O. Prakash and M. P. R. Prasad, “Design and Verification of Advanced Peripheral Bus (APB) RAM using System Verilog,” IEEE, 2022.

[13] I. H. Shanavas, P. Akhil Reddy, et.al “Design and Analysis of APB and AHB Lite Protocol,” in Proc. 2023 7th Int. Conf. Design Innovation for 3Cs Compute Communicate Control (ICDI3C), 2023

[14] G. Kumar and D. S. Yadav, “Design and Implementation of APB Interfaces for Complex SoC Environment,” in Proc. 2025 3rd World Conf. Communication & Computing (WCONF), 2025.

[15] M. S. M. Salman et.al “High Performance and Low Power AXI-Lite 4 Master Controller Using System Verilog,” in Proc. 2024 2nd Int. Conf. Artificial Intelligence and Machine Learning Applications (AIMLA), 2024.